g3_cpvass_sourcecode

July 23, 2026

1 Phase 1: Image Processing

This notebook completes Phase 1 for the Sentinel-2 Da Nang dataset.

The workflow now processes both exported AOI pieces (`above` and `below`) for both dates (2024-04-11 and 2026-04-11), computes NDVI from the Red (`B04`) and Near-Infrared (`B08`) bands, compares the two dates quantitatively and visually, and applies denoising plus contrast enhancement to true-color imagery.

All plots used in the report are exported to `../output`.

[1]:
```
import csv
from pathlib import Path

import cv2
import matplotlib.pyplot as plt
import numpy as np

# Paths are relative to this notebook when run from the works directory.
dataset_dir = Path("../dataset")
output_dir = Path("../output")
output_dir.mkdir(parents=True, exist_ok=True)

# Two Sentinel-2 dates selected in the same month to reduce seasonal effects.
dates = {
    "2024": {"folder": dataset_dir / "2024_04_11", "suffix": "2024", "label":␣
 ↪"April 11, 2024"},
    "2026": {"folder": dataset_dir / "2026_04_11", "suffix": "2026", "label":␣
 ↪"April 11, 2026"},
}

# The AOI was exported in two pieces, so Phase 1 evaluates both pieces.
crops = {
    "above": "Upper AOI crop",
    "below": "Lower AOI crop",
}

print(f"Dataset directory: {dataset_dir.resolve()}")
print(f"Output directory: {output_dir.resolve()}")
```

```
/home/dacekey/.config/matplotlib is not a writable directory

Matplotlib created a temporary cache directory at /tmp/matplotlib-rgvu00wm
because there was an issue with the default path
(/home/dacekey/.config/matplotlib); it is highly recommended to set the
MPLCONFIGDIR environment variable to a writable directory, in particular to
speed up the import of Matplotlib and to better support multiprocessing.

Dataset directory: /home/dacekey/CPV301_SUM26/main/L27_Assignment_*/dataset
Output directory: /home/dacekey/CPV301_SUM26/main/L27_Assignment_*/output
```

[2]:
```python
def find_one(folder, pattern):
    matches = sorted(folder.glob(pattern))
    if not matches:
        raise FileNotFoundError(f"No file matching {pattern!r} in {folder}")
    if len(matches) > 1:
        print(f"Warning: multiple matches for {pattern!r} in {folder}; using␣
 ↪{matches[0].name}")
    return matches[0]


def crop_folder(date_key, crop_key):
    config = dates[date_key]
    return config["folder"] / f"{crop_key}_{config['suffix']}"


def load_band(path):
    band = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if band is None:
        raise FileNotFoundError(f"Cannot load image: {path}")
    if band.ndim != 2:
        raise ValueError(f"Expected a single-band image for {path}, got shape␣
 ↪{band.shape}")
    return band.astype(np.float32)


def calculate_ndvi(red_band, nir_band):
    if red_band.shape != nir_band.shape:
        raise ValueError(f"Band shape mismatch: Red {red_band.shape}, NIR␣
 ↪{nir_band.shape}")
    denominator = nir_band + red_band
    return np.divide(
        nir_band - red_band,
        denominator,
        out=np.zeros_like(nir_band, dtype=np.float32),
        where=denominator != 0,
    )
```

```
def normalize_to_uint8(image):
    return cv2.normalize(image, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)


def load_true_color(path):
    image = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if image is None:
        raise FileNotFoundError(f"Cannot load image: {path}")
    if image.ndim != 3 or image.shape[2] < 3:
        raise ValueError(f"Expected a 3-channel true-color image for {path},␣
 ↪got shape {image.shape}")
    rgb = cv2.cvtColor(image[:, :, :3], cv2.COLOR_BGR2RGB)
    return normalize_to_uint8(rgb)


def apply_filter_pipeline(rgb_image):
    # Gaussian blur reduces high-frequency sensor noise but can soften sharp␣
 ↪boundaries.
    denoised = cv2.GaussianBlur(rgb_image, (5, 5), 0)

    # Equalize only the brightness channel to improve contrast while preserving␣
 ↪hue better.
    hsv = cv2.cvtColor(denoised, cv2.COLOR_RGB2HSV)
    h, s, v = cv2.split(hsv)
    v_equalized = cv2.equalizeHist(v)
    enhanced = cv2.cvtColor(cv2.merge((h, s, v_equalized)), cv2.COLOR_HSV2RGB)
    return denoised, enhanced


# def save_figure(fig, filename):
    path = output_dir / filename
#     fig.savefig(path, dpi=200, bbox_inches="tight")
    print(f"Saved: {path}")
    return path
```

[3]:
```
ndvi_data = {}
stats_rows = []

for crop_key, crop_label in crops.items():
    for date_key, date_info in dates.items():
        folder = crop_folder(date_key, crop_key)
        red_path = find_one(folder, "*B04*.tiff")
        nir_path = find_one(folder, "*B08*.tiff")

        red = load_band(red_path)
        nir = load_band(nir_path)
```

```
        ndvi = calculate_ndvi(red, nir)
        ndvi_data[(crop_key, date_key)] = ndvi

        stats_rows.append({
            "crop": crop_key,
            "date": date_key,
            "shape": f"{ndvi.shape[0]}x{ndvi.shape[1]}",
            "min": float(np.min(ndvi)),
            "max": float(np.max(ndvi)),
            "mean": float(np.mean(ndvi)),
            "std": float(np.std(ndvi)),
        })

stats_path = output_dir / "phase1_ndvi_statistics.csv"
with stats_path.open("w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["crop", "date", "shape", "min",␣
 ↪"max", "mean", "std"])
    writer.writeheader()
    writer.writerows(stats_rows)

print(f"Saved: {stats_path}")
print("\nNDVI summary statistics")
print(f"{'Crop':<8} {'Date':<6} {'Shape':<10} {'Min':>9} {'Max':>9} {'Mean':>9}␣
 ↪{'Std':>9}")
for row in stats_rows:
    print(
        f"{row['crop']:<8} {row['date']:<6} {row['shape']:<10} "
        f"{row['min']:>9.4f} {row['max']:>9.4f} {row['mean']:>9.4f} {row['std']:
 ↪>9.4f}"
    )
```

```
Saved: ../output/phase1_ndvi_statistics.csv

NDVI summary statistics
Crop     Date   Shape            Min       Max      Mean       Std
above    2024   252x774      -1.0000    0.8590    0.0711    0.6043
above    2026   252x774      -1.0000    0.8496    0.1769    0.4408
below    2024   313x743      -1.0000    0.8526    0.1114    0.5089
below    2026   313x743      -0.7572    0.8445    0.1887    0.3933
[ WARN:0@3.915] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@3.915] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@3.915] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@3.915] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
```

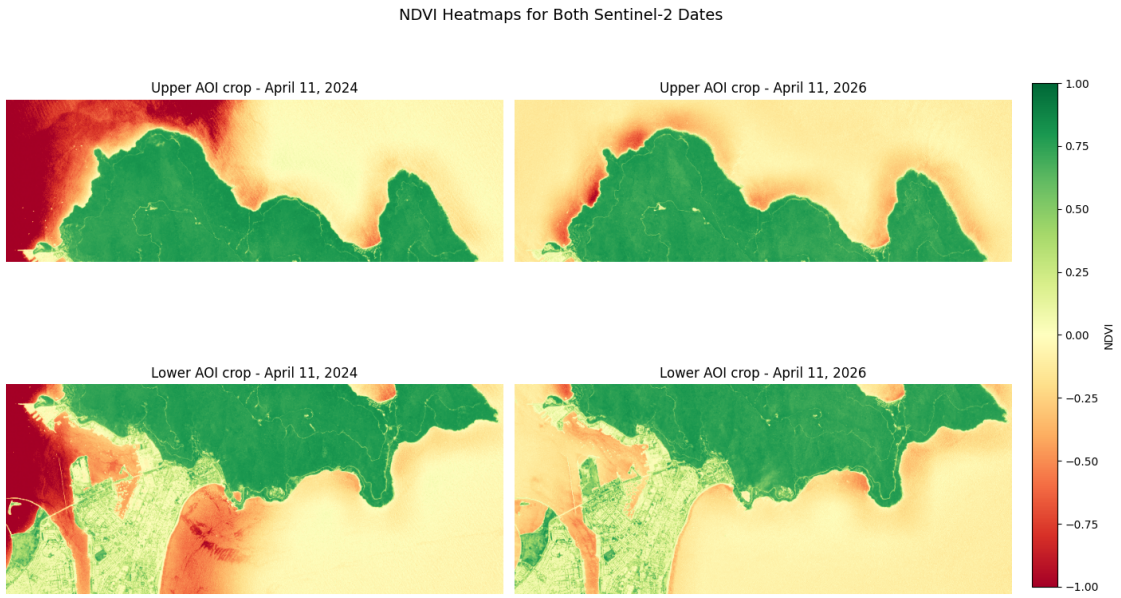
```
[ WARN:0@3.918] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@3.918] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@3.918] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@3.918] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@3.924] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@3.924] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@3.924] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@3.924] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@3.927] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@3.927] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@3.927] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@3.927] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@3.931] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@3.931] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@3.931] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@3.931] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@3.936] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@3.936] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@3.936] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@3.936] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@3.942] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@3.942] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@3.942] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@3.942] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
```

```
[ WARN:0@3.948] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@3.948] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@3.948] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@3.948] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
```

[4]:
```
fig, axes = plt.subplots(len(crops), len(dates), figsize=(14, 8),␣
 ↪constrained_layout=True)

for row_idx, (crop_key, crop_label) in enumerate(crops.items()):
    for col_idx, (date_key, date_info) in enumerate(dates.items()):
        ax = axes[row_idx, col_idx]
        im = ax.imshow(ndvi_data[(crop_key, date_key)], cmap="RdYlGn", vmin=-1,␣
 ↪vmax=1)
        ax.set_title(f"{crop_label} - {date_info['label']}")
        ax.axis("off")

fig.colorbar(im, ax=axes.ravel().tolist(), fraction=0.025, pad=0.02,␣
 ↪label="NDVI")
fig.suptitle("NDVI Heatmaps for Both Sentinel-2 Dates", fontsize=14)
# save_figure(fig, "phase1_ndvi_heatmaps.png")
plt.show()
```



[5]:
```
fig, axes = plt.subplots(1, len(crops), figsize=(14, 5),␣
 ↪constrained_layout=True)
mean_changes = []

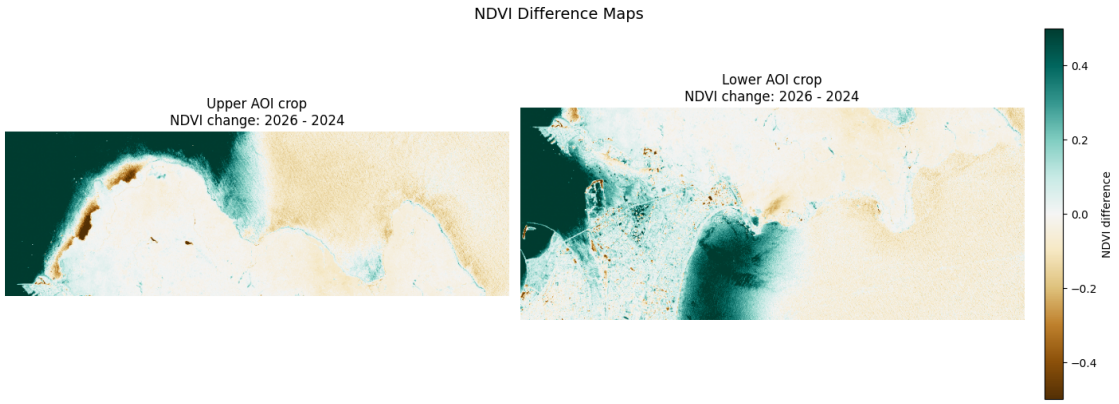
for ax, (crop_key, crop_label) in zip(axes, crops.items()):
    ndvi_2024 = ndvi_data[(crop_key, "2024")]
    ndvi_2026 = ndvi_data[(crop_key, "2026")]
    if ndvi_2024.shape != ndvi_2026.shape:
        raise ValueError(f"Cannot compare dates for {crop_key}: {ndvi_2024.
 ↪shape} vs {ndvi_2026.shape}")

    diff = ndvi_2026 - ndvi_2024
    mean_delta = float(np.mean(diff))
    mean_changes.append((crop_key, mean_delta))

    im = ax.imshow(diff, cmap="BrBG", vmin=-0.5, vmax=0.5)
    ax.set_title(f"{crop_label}\nNDVI change: 2026 - 2024")
    ax.axis("off")

fig.colorbar(im, ax=axes.ravel().tolist(), fraction=0.035, pad=0.02,␣
 ↪label="NDVI difference")
fig.suptitle("NDVI Difference Maps", fontsize=14)
# save_figure(fig, "phase1_ndvi_difference.png")
plt.show()

print("Mean NDVI change by AOI crop")
for crop_key, mean_delta in mean_changes:
    direction = "increase" if mean_delta > 0 else "decrease" if mean_delta < 0␣
 ↪else "no net change"
    print(f"- {crop_key}: {mean_delta:+.4f} ({direction})")
```



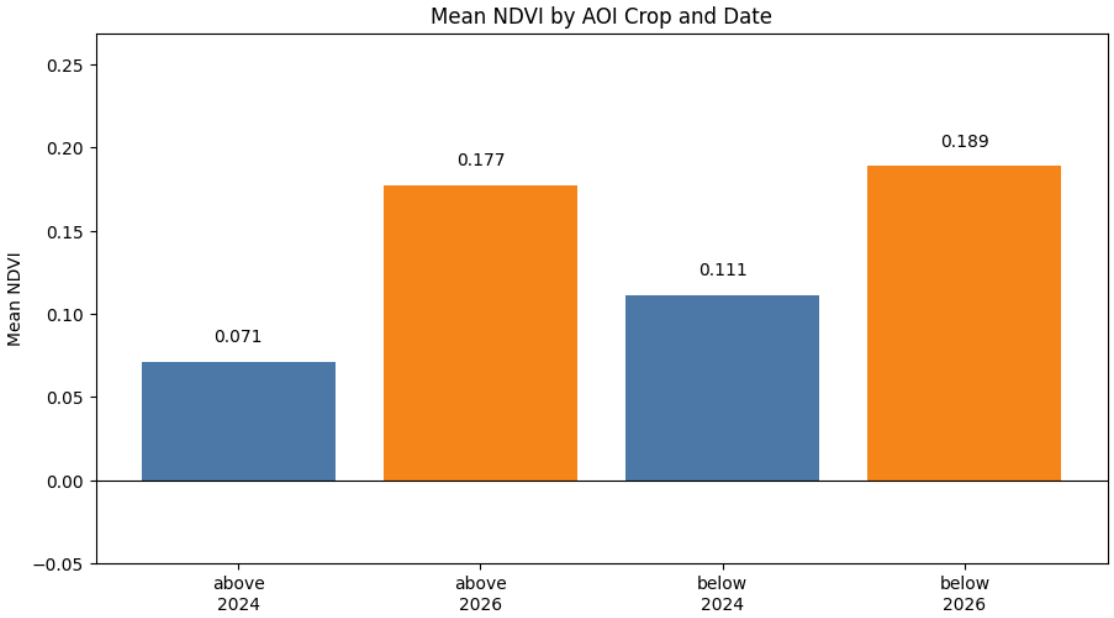

```
Mean NDVI change by AOI crop
- above: +0.1059 (increase)
```

- below: +0.0773 (increase)

[6]:
```
labels = []
means = []
colors = []
for crop_key, crop_label in crops.items():
    for date_key in dates:
        row = next(r for r in stats_rows if r["crop"] == crop_key and r["date"]
 ↪== date_key)
        labels.append(f"{crop_key}\n{date_key}")
        means.append(row["mean"])
        colors.append("#4c78a8" if date_key == "2024" else "#f58518")

fig, ax = plt.subplots(figsize=(9, 5), constrained_layout=True)
bars = ax.bar(labels, means, color=colors)
ax.axhline(0, color="black", linewidth=0.8)
ax.set_ylabel("Mean NDVI")
ax.set_title("Mean NDVI by AOI Crop and Date")
ax.set_ylim(min(-0.05, min(means) - 0.05), max(means) + 0.08)
for bar, value in zip(bars, means):
    ax.text(bar.get_x() + bar.get_width() / 2, value + 0.01, f"{value:.3f}",
 ↪ha="center", va="bottom")

# save_figure(fig, "phase1_ndvi_mean_comparison.png")
plt.show()
```



1.1 Denoising and Contrast Enhancement

Gaussian blur is used as the denoising/smoothing filter because it suppresses small high-frequency sensor variation in the true-color image. The main trade-off is that very thin boundaries such as roads, coastlines, and small construction edges can become slightly softer.

Histogram equalization is used for contrast enhancement. It is applied to the HSV value channel instead of directly to RGB channels, which improves brightness contrast while reducing color distortion. The trade-off is that weak noise or compression artifacts may become more visible in low-texture regions.

[7]:
```
filter_results = {}

for crop_key, crop_label in crops.items():
    folder = crop_folder("2024", crop_key)
    true_color_path = find_one(folder, "*True_color*.tiff")
    original = load_true_color(true_color_path)
    denoised, enhanced = apply_filter_pipeline(original)
    filter_results[crop_key] = {
        "original": original,
        "denoised": denoised,
        "enhanced": enhanced,
    }

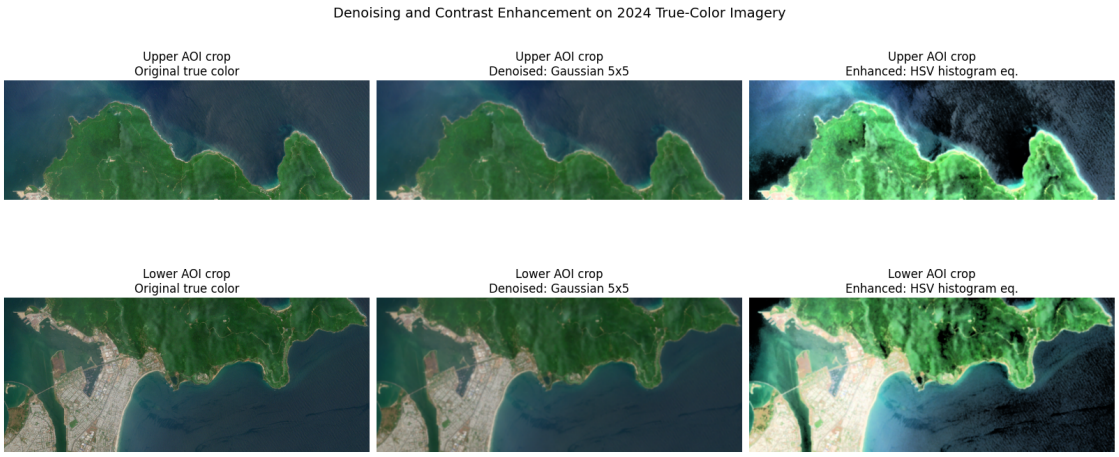
fig, axes = plt.subplots(len(crops), 3, figsize=(16, 7),␣
 ↪constrained_layout=True)
columns = [
    ("original", "Original true color"),
    ("denoised", "Denoised: Gaussian 5x5"),
    ("enhanced", "Enhanced: HSV histogram eq."),
]

for row_idx, (crop_key, crop_label) in enumerate(crops.items()):
    for col_idx, (image_key, title) in enumerate(columns):
        ax = axes[row_idx, col_idx]
        ax.imshow(filter_results[crop_key][image_key])
        ax.set_title(f"{crop_label}\n{title}")
        ax.axis("off")

fig.suptitle("Denoising and Contrast Enhancement on 2024 True-Color Imagery",␣
 ↪fontsize=14)
# save_figure(fig, "phase1_filter_contrast_comparison.png")
plt.show()
```

```
[ WARN:0@5.179] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@5.179] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@5.179] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
```

```
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@5.179] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@5.192] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@5.192] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@5.192] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@5.192] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
```



1.2 Phase 1 Notes

The NDVI maps use the required point operation on matching Red and NIR bands for both acquisition dates. The notebook checks band shape consistency before computing NDVI, reports min/max/mean/std values, and exports all plots for the written report.

The two AOI pieces are not stitched in Phase 1 because stitching belongs to Phase 3. Therefore, the upper and lower crops are processed and compared independently here.

2 Phase 2: Segmentation and Features

This notebook implements Phase 2 for the Da Nang Sentinel-2 assignment.

Manual requirements covered here: 1. Detect edges/features in the scene using Canny and ORB. 2. Segment the scene into land-cover regions using OpenCV mean-shift filtering plus spectral-index interpretation. 3. Validate 5 selected regions against a basemap-style reference and report where the segmentation matches or is limited.

The AOI was exported as two overlapping pieces (`above` and `below`). Phase 2 processes both pieces independently on the 2024 reference date; stitching/alignment is reserved for Phase 3.

[8]:
```
import csv
import math
import urllib.request
from pathlib import Path

import cv2
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from matplotlib.patches import Patch, Rectangle

# Paths are relative to this notebook when run from the works directory.
dataset_dir = Path("../dataset")
output_dir = Path("../output")
output_dir.mkdir(parents=True, exist_ok=True)

reference_year = "2024"
reference_date_label = "April 11, 2024"
reference_dir = dataset_dir / "2024_04_11"

crops = {
    "above": "Upper AOI crop",
    "below": "Lower AOI crop",
}

print(f"Dataset directory: {dataset_dir.resolve()}")
print(f"Output directory: {output_dir.resolve()}")
```

```
Dataset directory: /home/dacekey/CPV301_SUM26/main/L27_Assignment_*/dataset
Output directory: /home/dacekey/CPV301_SUM26/main/L27_Assignment_*/output
```

[9]:
```
def find_one(folder, pattern):
    matches = sorted(folder.glob(pattern))
    if not matches:
        raise FileNotFoundError(f"No file matching {pattern!r} in {folder}")
    if len(matches) > 1:
        print(f"Warning: multiple matches for {pattern!r} in {folder}; using␣
 ↪{matches[0].name}")
    return matches[0]


def crop_folder(crop_key):
    return reference_dir / f"{crop_key}_{reference_year}"


def normalize_to_uint8(image):
    return cv2.normalize(image, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)
```

```
def load_true_color(path):
    image = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if image is None:
        raise FileNotFoundError(f"Cannot load image: {path}")
    if image.ndim != 3 or image.shape[2] < 3:
        raise ValueError(f"Expected a 3-channel true-color image for {path},␣
 ↪got shape {image.shape}")
    rgb = cv2.cvtColor(image[:, :, :3], cv2.COLOR_BGR2RGB)
    return normalize_to_uint8(rgb)


def load_band(path):
    band = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if band is None:
        raise FileNotFoundError(f"Cannot load image: {path}")
    if band.ndim != 2:
        raise ValueError(f"Expected single-band TIFF for {path}, got shape␣
 ↪{band.shape}")
    return band.astype(np.float32)


def safe_index(numerator, denominator):
    return np.divide(
        numerator,
        denominator,
        out=np.zeros_like(numerator, dtype=np.float32),
        where=denominator != 0,
    )


def calculate_ndvi(red, nir):
    if red.shape != nir.shape:
        raise ValueError(f"Band shape mismatch: Red {red.shape}, NIR {nir.
 ↪shape}")
    return safe_index(nir - red, nir + red)


def calculate_ndwi(green, nir):
    if green.shape != nir.shape:
        raise ValueError(f"Band shape mismatch: Green {green.shape}, NIR {nir.
 ↪shape}")
    return safe_index(green - nir, green + nir)


def auto_canny(gray, sigma=0.33):
```

```
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
    median = float(np.median(blurred))
    lower = int(max(0, (1.0 - sigma) * median))
    upper = int(min(255, (1.0 + sigma) * median))
    edges = cv2.Canny(blurred, lower, upper)
    return edges, lower, upper


# def save_figure(fig, filename):
    path = output_dir / filename
#     fig.savefig(path, dpi=200, bbox_inches="tight")
    print(f"Saved: {path}")
    return path


def read_georef(path):
    # Sentinel Hub GeoTIFF exports store WGS84 tiepoint and pixel scale tags.
    image = Image.open(path)
    tags = image.tag_v2
    tiepoint = tags.get(33922)
    scale = tags.get(33550)
    if tiepoint is None or scale is None:
        raise ValueError(f"Missing GeoTIFF tiepoint/pixel-scale tags in {path}")
    lon0 = float(tiepoint[3])
    lat0 = float(tiepoint[4])
    scale_x = float(scale[0])
    scale_y = float(scale[1])
    width, height = image.size
    return {
        "lon0": lon0,
        "lat0": lat0,
        "scale_x": scale_x,
        "scale_y": scale_y,
        "width": width,
        "height": height,
        "lon_min": lon0,
        "lon_max": lon0 + width * scale_x,
        "lat_max": lat0,
        "lat_min": lat0 - height * scale_y,
    }


def pixel_to_lonlat(georef, x, y):
    lon = georef["lon0"] + x * georef["scale_x"]
    lat = georef["lat0"] - y * georef["scale_y"]
    return lon, lat
```

[10]:
```
scene_data = {}

for crop_key, crop_label in crops.items():
    folder = crop_folder(crop_key)
    true_color_path = find_one(folder, "*True_color*.tiff")
    red_path = find_one(folder, "*B04*.tiff")
    green_path = find_one(folder, "*B03*.tiff")
    nir_path = find_one(folder, "*B08*.tiff")

    rgb = load_true_color(true_color_path)
    red = load_band(red_path)
    green = load_band(green_path)
    nir = load_band(nir_path)

    if rgb.shape[:2] != red.shape:
        raise ValueError(f"True-color and band shape mismatch for {crop_key}:␣
 ↪{rgb.shape[:2]} vs {red.shape}")

    ndvi = calculate_ndvi(red, nir)
    ndwi = calculate_ndwi(green, nir)
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
    georef = read_georef(red_path)

    scene_data[crop_key] = {
        "label": crop_label,
        "rgb": rgb,
        "gray": gray,
        "red": red,
        "green": green,
        "nir": nir,
        "ndvi": ndvi,
        "ndwi": ndwi,
        "georef": georef,
    }

    print(
        f"{crop_label}: rgb={rgb.shape}, bands={red.shape}, "
        f"lon=({georef['lon_min']:.6f}, {georef['lon_max']:.6f}), "
        f"lat=({georef['lat_min']:.6f}, {georef['lat_max']:.6f})"
    )
```

```
Upper AOI crop: rgb=(252, 774, 3), bands=(252, 774), lon=(108.205719,
108.344765), lat=(16.119543, 16.163075)
Lower AOI crop: rgb=(313, 743, 3), bands=(313, 743), lon=(108.205719,
108.339272), lat=(16.073692, 16.127788)
```

```
[ WARN:0@5.842] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
```

```
[ WARN:0@5.842] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@5.842] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@5.842] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@5.848] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@5.848] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@5.848] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@5.848] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@5.852] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@5.852] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@5.852] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@5.852] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@5.855] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@5.855] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@5.855] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@5.855] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@5.864] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@5.864] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@5.864] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@5.864] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@5.872] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@5.872] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@5.872] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@5.872] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@5.875] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
```

```
[ WARN:0@5.875] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@5.875] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@5.875] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@5.878] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@5.878] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@5.878] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@5.878] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
```

[11]:

```
feature_rows = []
edge_results = {}
orb_results = {}

orb = cv2.ORB_create(nfeatures=800, scaleFactor=1.2, nlevels=8,␣
 ↪edgeThreshold=15)

fig_edges, axes_edges = plt.subplots(len(crops), 3, figsize=(16, 7),␣
 ↪constrained_layout=True)
fig_orb, axes_orb = plt.subplots(len(crops), 1, figsize=(14, 7),␣
 ↪constrained_layout=True)

for row_idx, (crop_key, crop_label) in enumerate(crops.items()):
    rgb = scene_data[crop_key]["rgb"]
    gray = scene_data[crop_key]["gray"]

    edges, lower, upper = auto_canny(gray)
    edge_overlay = rgb.copy()
    edge_overlay[edges > 0] = [255, 0, 0]

    keypoints = orb.detect(gray, None)
    keypoints = sorted(keypoints, key=lambda kp: kp.response, reverse=True)[:
 ↪250]
    keypoint_image = cv2.drawKeypoints(
        rgb,
        keypoints,
        None,
        color=(0, 255, 0),
        flags=cv2.DrawMatchesFlags_DRAW_RICH_KEYPOINTS,
    )

    edge_results[crop_key] = edges
```

```
    orb_results[crop_key] = keypoints

    edge_count = int(np.count_nonzero(edges))
    edge_density = edge_count / edges.size
    feature_rows.append({
        "crop": crop_key,
        "canny_low": lower,
        "canny_high": upper,
        "edge_pixels": edge_count,
        "edge_density": edge_density,
        "orb_keypoints": len(keypoints),
    })

    axes_edges[row_idx, 0].imshow(rgb)
    axes_edges[row_idx, 0].set_title(f"{crop_label}\nTrue color")
    axes_edges[row_idx, 1].imshow(edges, cmap="gray")
    axes_edges[row_idx, 1].set_title(f"Canny edges\nthresholds {lower}/{upper}")
    axes_edges[row_idx, 2].imshow(edge_overlay)
    axes_edges[row_idx, 2].set_title("Edges overlaid in red")
    for col_idx in range(3):
        axes_edges[row_idx, col_idx].axis("off")

    axes_orb[row_idx].imshow(keypoint_image)
    axes_orb[row_idx].set_title(f"{crop_label}: top {len(keypoints)} ORB␣
 ↪keypoints")
    axes_orb[row_idx].axis("off")

feature_summary_path = output_dir / "phase2_feature_summary.csv"
with feature_summary_path.open("w", newline="") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["crop", "canny_low", "canny_high", "edge_pixels",␣
 ↪"edge_density", "orb_keypoints"],
    )
    writer.writeheader()
    writer.writerows(feature_rows)

print(f"Saved: {feature_summary_path}")
print("\nFeature detection summary")
for row in feature_rows:
    print(
        f"- {row['crop']}: Canny {row['canny_low']}/{row['canny_high']}, "
        f"edge density={row['edge_density']:.4f}, ORB␣
 ↪keypoints={row['orb_keypoints']}"
    )

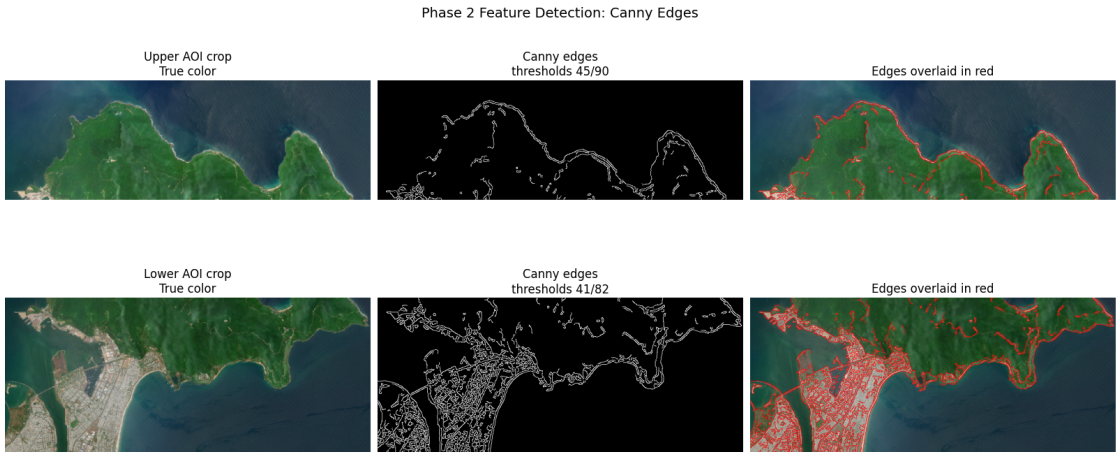
fig_edges.suptitle("Phase 2 Feature Detection: Canny Edges", fontsize=14)
```

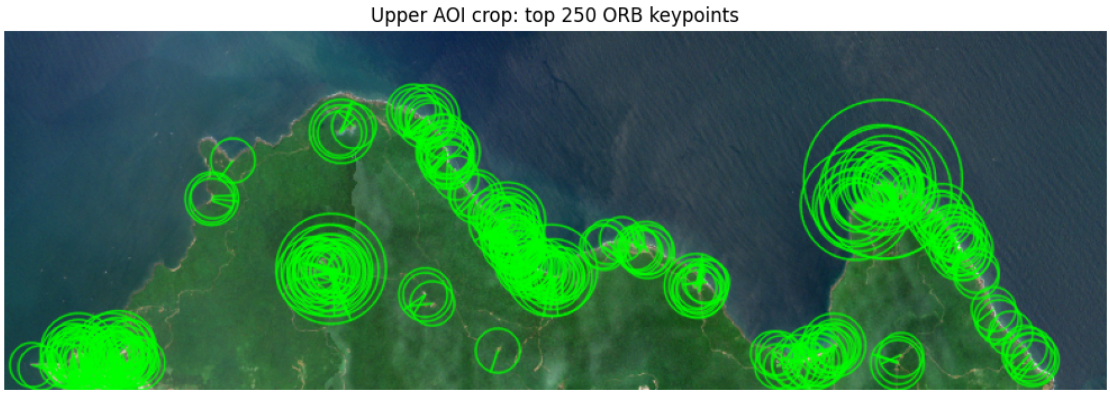
```
# save_figure(fig_edges, "phase2_canny_edges_2024.png")
plt.show()

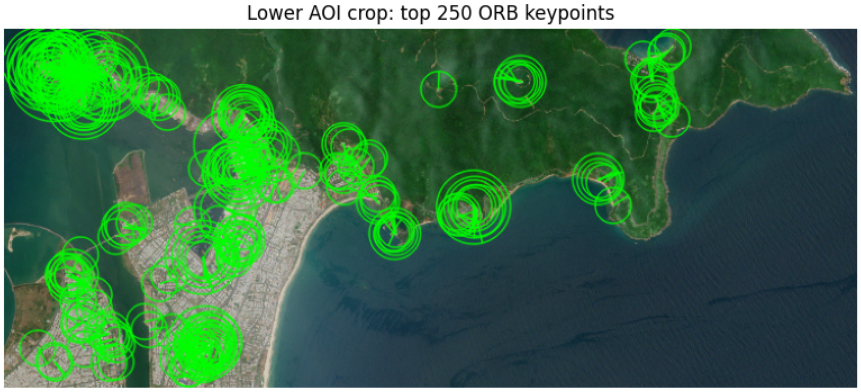
fig_orb.suptitle("Phase 2 Feature Detection: ORB Keypoints", fontsize=14)
# save_figure(fig_orb, "phase2_orb_keypoints_2024.png")
plt.show()
```

```
Saved: ../output/phase2_feature_summary.csv

Feature detection summary
- above: Canny 45/90, edge density=0.0241, ORB keypoints=250
- below: Canny 41/82, edge density=0.0700, ORB keypoints=250
```







[12]:
```
label_names = {
    0: "Water",
    1: "Dense vegetation",
    2: "Built-up/bare",
    3: "Sand/exposed soil",
    4: "Mixed/shadow",
}

label_colors = np.array([
    [37, 99, 235],      # water: blue
    [34, 139, 34],      # vegetation: green
    [220, 38, 38],      # built-up/bare: red
    [245, 158, 11],     # sand/exposed soil: orange
    [107, 114, 128],    # mixed/shadow: gray
], dtype=np.uint8)

segmentation_results = {}
area_rows = []

fig, axes = plt.subplots(len(crops), 4, figsize=(18, 8),␣
 ↪constrained_layout=True)
```

```
for row_idx, (crop_key, crop_label) in enumerate(crops.items()):
    rgb = scene_data[crop_key]["rgb"]
    ndvi = scene_data[crop_key]["ndvi"]
    ndwi = scene_data[crop_key]["ndwi"]

    # Mean shift groups spatially close pixels with similar colors. The␣
 ↪spectral masks below
    # turn those smoothed regions into interpretable land-cover classes.
    mean_shift = cv2.pyrMeanShiftFiltering(rgb, sp=12, sr=25, maxLevel=1)
    ms_gray = cv2.cvtColor(mean_shift, cv2.COLOR_RGB2GRAY)
    ms_hsv = cv2.cvtColor(mean_shift, cv2.COLOR_RGB2HSV)
    ms_saturation = ms_hsv[:, :, 1]

    labels = np.full(ndvi.shape, 4, dtype=np.uint8)
    water = ((ndwi > 0.05) & (ndvi < 0.10)) | ((ms_gray < 70) & (ndvi < 0.15))
    vegetation = ndvi > 0.35
    built = (~water) & (~vegetation) & (ndvi < 0.25) & (ms_gray > 115)
    sand = (~water) & (~vegetation) & (~built) & (ndvi > -0.25) & (ndvi < 0.20)␣
 ↪& (ms_gray > 80) & (ms_saturation < 100)

    labels[water] = 0
    labels[vegetation] = 1
    labels[built] = 2
    labels[sand] = 3

    label_image = label_colors[labels]
    overlay = (0.58 * rgb + 0.42 * label_image).astype(np.uint8)

    segmentation_results[crop_key] = {
        "mean_shift": mean_shift,
        "labels": labels,
        "label_image": label_image,
        "overlay": overlay,
    }

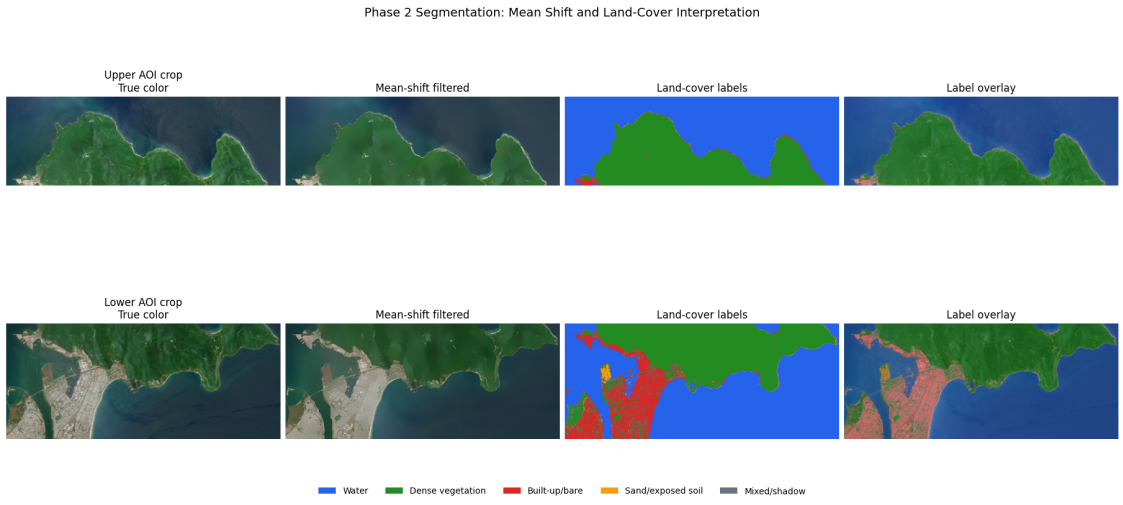
    unique, counts = np.unique(labels, return_counts=True)
    total = labels.size
    for label_id, count in zip(unique, counts):
        area_rows.append({
            "crop": crop_key,
            "class_id": int(label_id),
            "class_name": label_names[int(label_id)],
            "pixels": int(count),
            "percent": float(count / total * 100.0),
        })
```

```
    axes[row_idx, 0].imshow(rgb)
    axes[row_idx, 0].set_title(f"{crop_label}\nTrue color")
    axes[row_idx, 1].imshow(mean_shift)
    axes[row_idx, 1].set_title("Mean-shift filtered")
    axes[row_idx, 2].imshow(label_image)
    axes[row_idx, 2].set_title("Land-cover labels")
    axes[row_idx, 3].imshow(overlay)
    axes[row_idx, 3].set_title("Label overlay")
    for col_idx in range(4):
        axes[row_idx, col_idx].axis("off")

legend_handles = [Patch(facecolor=label_colors[i] / 255.0,␣
 ↪label=label_names[i]) for i in sorted(label_names)]
fig.legend(handles=legend_handles, loc="lower center", ncol=len(label_names),␣
 ↪frameon=False)
fig.suptitle("Phase 2 Segmentation: Mean Shift and Land-Cover Interpretation",␣
 ↪fontsize=14)
# save_figure(fig, "phase2_mean_shift_landcover_segmentation_2024.png")
plt.show()

area_summary_path = output_dir / "phase2_landcover_area_summary.csv"
with area_summary_path.open("w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["crop", "class_id", "class_name",␣
 ↪"pixels", "percent"])
    writer.writeheader()
    writer.writerows(area_rows)

print(f"Saved: {area_summary_path}")
print("\nLand-cover area summary")
for row in area_rows:
    print(f"- {row['crop']} / {row['class_name']}: {row['percent']:.2f}%")
```



```
Saved: ../output/phase2_landcover_area_summary.csv

Land-cover area summary
- above / Water: 61.96%
- above / Dense vegetation: 36.72%
- above / Built-up/bare: 0.87%
- above / Sand/exposed soil: 0.14%
- above / Mixed/shadow: 0.30%
- below / Water: 52.50%
- below / Dense vegetation: 32.81%
- below / Built-up/bare: 11.29%
- below / Sand/exposed soil: 0.69%
- below / Mixed/shadow: 2.70%
```

[13]:
```
validation_regions = [
    {
        "id": "R1",
        "crop": "above",
        "roi": (220, 90, 100, 80),
        "expected_land_use": "Forested Son Tra hillside",
        "acceptable_labels": ["Dense vegetation"],
        "notes": "Should be mostly vegetation; small edge/water pixels can␣
↪appear near steep coastal boundaries.",
    },
    {
        "id": "R2",
        "crop": "above",
        "roi": (420, 20, 120, 70),
        "expected_land_use": "Open coastal water",
        "acceptable_labels": ["Water"],
        "notes": "Dark offshore water is expected; this region tests whether␣
↪water is separated from shadows.",
    },
    {
        "id": "R3",
        "crop": "below",
        "roi": (130, 180, 160, 80),
        "expected_land_use": "Dense urban/built-up grid",
        "acceptable_labels": ["Built-up/bare"],
        "notes": "Urban blocks include roads, rooftops, vegetation, and␣
↪shadows, so mixed pixels are expected.",
    },
    {
        "id": "R4",
        "crop": "below",
        "roi": (250, 190, 80, 60),
```

```
        "expected_land_use": "Beach and nearshore water corridor",
        "acceptable_labels": ["Water", "Sand/exposed soil"],
        "notes": "A 10 m Sentinel-2 pixel can mix beach sand, surf, shallow␣
 ↪water, and built edges.",
    },
    {
        "id": "R5",
        "crop": "below",
        "roi": (20, 120, 75, 55),
        "expected_land_use": "Harbor or bay water",
        "acceptable_labels": ["Water"],
        "notes": "Sediment and shallow water may reduce NDVI/NDWI separation␣
 ↪near the shoreline.",
    },
]

validation_rows = []

fig, axes = plt.subplots(len(crops), 2, figsize=(14, 9),␣
 ↪constrained_layout=True)
region_colors = ["yellow", "cyan", "magenta", "white", "orange"]
region_color_map = {region["id"]: region_colors[idx % len(region_colors)] for␣
 ↪idx, region in enumerate(validation_regions)}

for row_idx, (crop_key, crop_label) in enumerate(crops.items()):
    rgb = scene_data[crop_key]["rgb"]
    label_image = segmentation_results[crop_key]["label_image"]
    labels = segmentation_results[crop_key]["labels"]
    georef = scene_data[crop_key]["georef"]

    axes[row_idx, 0].imshow(rgb)
    axes[row_idx, 0].set_title(f"{crop_label}\nValidation ROIs on true color")
    axes[row_idx, 1].imshow(label_image)
    axes[row_idx, 1].set_title("Validation ROIs on land-cover labels")

    for ax in axes[row_idx]:
        ax.axis("off")

    for region in [r for r in validation_regions if r["crop"] == crop_key]:
        x, y, w, h = region["roi"]
        roi_labels = labels[y:y+h, x:x+w]
        unique, counts = np.unique(roi_labels, return_counts=True)
        order = np.argsort(counts)[::-1]
        majority_id = int(unique[order[0]])
        majority_label = label_names[majority_id]
        majority_percent = float(counts[order[0]] / roi_labels.size * 100.0)
        accepted = majority_label in region["acceptable_labels"]
```

```
        status = "Match" if accepted else "Needs review"
        lon, lat = pixel_to_lonlat(georef, x + w / 2, y + h / 2)

        validation_rows.append({
            "region_id": region["id"],
            "crop": crop_key,
            "roi_x": x,
            "roi_y": y,
            "roi_w": w,
            "roi_h": h,
            "center_lon": lon,
            "center_lat": lat,
            "expected_land_use": region["expected_land_use"],
            "acceptable_labels": "; ".join(region["acceptable_labels"]),
            "majority_label": majority_label,
            "majority_percent": majority_percent,
            "status": status,
            "notes": region["notes"],
        })

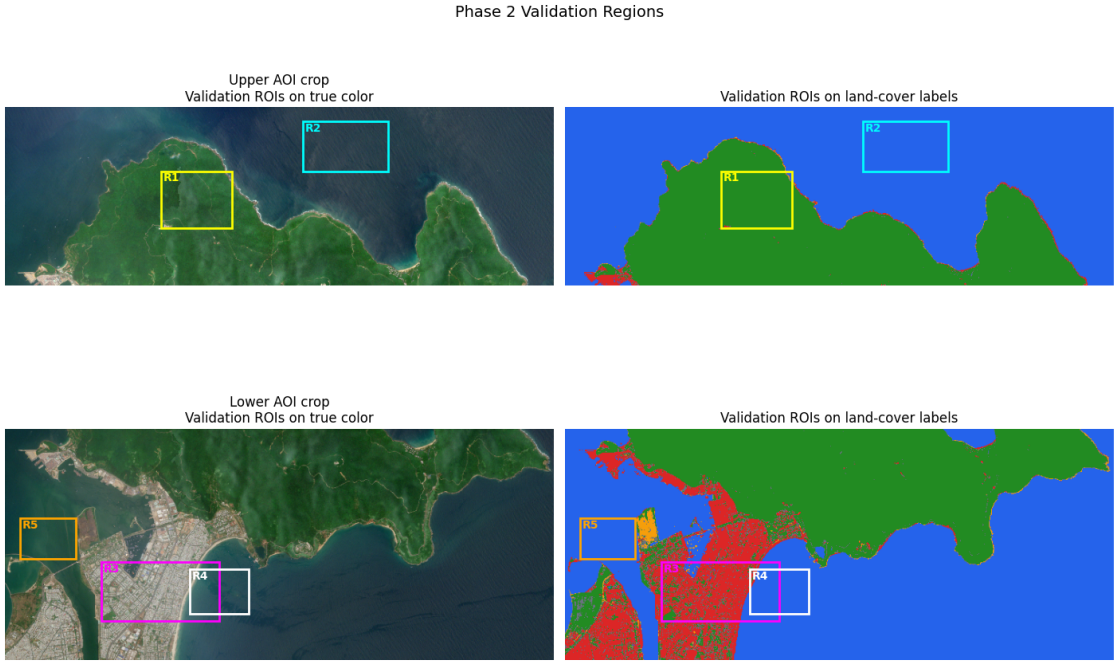
        color = region_color_map[region["id"]]
        for ax in axes[row_idx]:
            rect = Rectangle((x, y), w, h, fill=False, edgecolor=color,␣
 ↪linewidth=2.0)
            ax.add_patch(rect)
            ax.text(x + 3, y + 13, region["id"], color=color, fontsize=10,␣
 ↪weight="bold")

validation_path = output_dir / "phase2_validation_regions.csv"
with validation_path.open("w", newline="") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=[
            "region_id", "crop", "roi_x", "roi_y", "roi_w", "roi_h",␣
 ↪"center_lon", "center_lat",
            "expected_land_use", "acceptable_labels", "majority_label",␣
 ↪"majority_percent", "status", "notes",
        ],
    )
    writer.writeheader()
    writer.writerows(validation_rows)

fig.suptitle("Phase 2 Validation Regions", fontsize=14)
# save_figure(fig, "phase2_validation_regions_2024.png")
plt.show()

print(f"Saved: {validation_path}")
```

```
print("\nValidation summary")
for row in validation_rows:
    print(
        f"- {row['region_id']} ({row['crop']}): expected␣
 ↪{row['expected_land_use']}; "
        f"majority={row['majority_label']} ({row['majority_percent']:.1f}%) ->␣
 ↪{row['status']}"
    )
```




```
Saved: ../output/phase2_validation_regions.csv

Validation summary
- R1 (above): expected Forested Son Tra hillside; majority=Dense vegetation
(98.4%) -> Match
- R2 (above): expected Open coastal water; majority=Water (100.0%) -> Match
- R3 (below): expected Dense urban/built-up grid; majority=Built-up/bare (56.5%)
-> Match
- R4 (below): expected Beach and nearshore water corridor; majority=Water
(96.0%) -> Match
- R5 (below): expected Harbor or bay water; majority=Water (100.0%) -> Match
```

[14]:
```
def lonlat_to_tile(lon, lat, zoom):
    lat = max(min(lat, 85.05112878), -85.05112878)
    lat_rad = math.radians(lat)
    n = 2 ** zoom
```

```
    x_tile = int((lon + 180.0) / 360.0 * n)
    y_tile = int((1.0 - math.asinh(math.tan(lat_rad)) / math.pi) / 2.0 * n)
    return x_tile, y_tile


def tile_bounds(x_tile, y_tile, zoom):
    n = 2 ** zoom
    lon_w = x_tile / n * 360.0 - 180.0
    lon_e = (x_tile + 1) / n * 360.0 - 180.0
    lat_n = math.degrees(math.atan(math.sinh(math.pi * (1 - 2 * y_tile / n))))
    lat_s = math.degrees(math.atan(math.sinh(math.pi * (1 - 2 * (y_tile + 1) /␣
 ↪n))))
    return lon_w, lat_s, lon_e, lat_n


def fetch_osm_tile(x_tile, y_tile, zoom):
    url = f"https://tile.openstreetmap.org/{zoom}/{x_tile}/{y_tile}.png"
    request = urllib.request.Request(url, headers={"User-Agent":␣
 ↪"CPV301-phase2-validation/1.0"})
    with urllib.request.urlopen(request, timeout=15) as response:
        return Image.open(response).convert("RGB")

lon_min = min(scene_data[crop]["georef"]["lon_min"] for crop in crops)
lon_max = max(scene_data[crop]["georef"]["lon_max"] for crop in crops)
lat_min = min(scene_data[crop]["georef"]["lat_min"] for crop in crops)
lat_max = max(scene_data[crop]["georef"]["lat_max"] for crop in crops)

zoom = 13
x_min, y_max = lonlat_to_tile(lon_min, lat_min, zoom)
x_max, y_min = lonlat_to_tile(lon_max, lat_max, zoom)

try:
    tile_size = 256
    mosaic = Image.new("RGB", ((x_max - x_min + 1) * tile_size, (y_max - y_min␣
 ↪+ 1) * tile_size))
    for x_tile in range(x_min, x_max + 1):
        for y_tile in range(y_min, y_max + 1):
            tile = fetch_osm_tile(x_tile, y_tile, zoom)
            mosaic.paste(tile, ((x_tile - x_min) * tile_size, (y_tile - y_min)␣
 ↪* tile_size))

    tile_lon_w, tile_lat_s, _, _ = tile_bounds(x_min, y_max, zoom)
    _, _, tile_lon_e, tile_lat_n = tile_bounds(x_max, y_min, zoom)

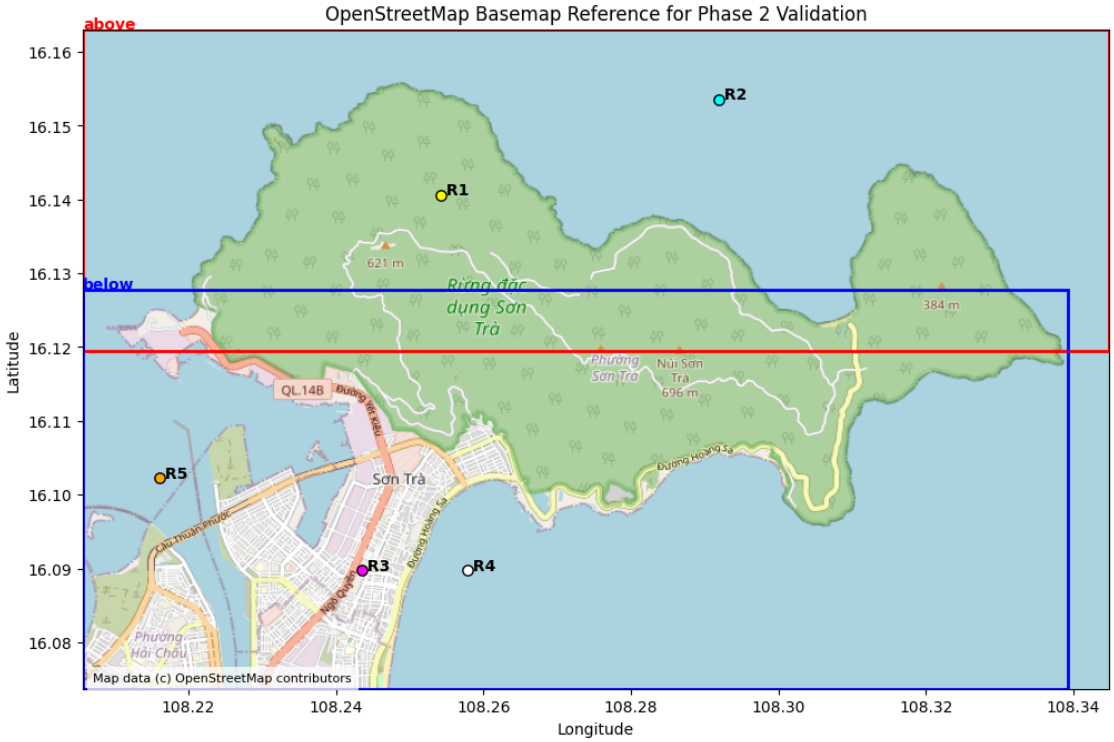
    fig, ax = plt.subplots(figsize=(10, 8), constrained_layout=True)
    ax.imshow(mosaic, extent=[tile_lon_w, tile_lon_e, tile_lat_s, tile_lat_n])
    ax.set_xlim(lon_min, lon_max)
```

```
    ax.set_ylim(lat_min, lat_max)
    ax.set_xlabel("Longitude")
    ax.set_ylabel("Latitude")
    ax.set_title("OpenStreetMap Basemap Reference for Phase 2 Validation")

    for crop_key, crop_label in crops.items():
        georef = scene_data[crop_key]["georef"]
        rect = Rectangle(
            (georef["lon_min"], georef["lat_min"]),
            georef["lon_max"] - georef["lon_min"],
            georef["lat_max"] - georef["lat_min"],
            fill=False,
            edgecolor="red" if crop_key == "above" else "blue",
            linewidth=2,
        )
        ax.add_patch(rect)
        ax.text(georef["lon_min"], georef["lat_max"], crop_key, color=rect.
↪get_edgecolor(), weight="bold")

    for region in validation_rows:
        ax.scatter(region["center_lon"], region["center_lat"], s=45,␣
↪color=region_color_map[region["region_id"]], edgecolor="black")
        ax.text(region["center_lon"], region["center_lat"], " " +␣
↪region["region_id"], color="black", weight="bold")

    ax.text(
        0.01,
        0.01,
        "Map data (c) OpenStreetMap contributors",
        transform=ax.transAxes,
        fontsize=8,
        color="black",
        bbox={"facecolor": "white", "alpha": 0.75, "edgecolor": "none"},
    )
#     save_figure(fig, "phase2_osm_basemap_reference.png")
    plt.show()
except Exception as exc:
    print(f"OSM basemap fetch skipped: {exc}")
    print("Validation CSV still contains ROI center lon/lat values for manual␣
↪basemap checking.")
```



2.1 Phase 2 Notes

Canny highlights coastlines, roads, riverbanks, and urban boundaries as edge structures. ORB keypoints concentrate around high-contrast corners and texture changes, especially urban grids, coastline curves, and forest-road boundaries.

Mean-shift filtering is used as the segmentation technique required by Phase 2. The filtered image groups nearby pixels with similar color before the notebook assigns interpretable land-cover classes using NDVI, NDWI, brightness, and saturation thresholds.

The validation regions are intentionally small and inspectable. They should be reviewed against Google Maps or an OpenStreetMap basemap for the same coordinates before the final written report. The notebook exports an OpenStreetMap reference image when internet access is available and writes ROI center coordinates to `phase2_validation_regions.csv`.

3 Phase 3: Alignment and Stitching

This notebook implements Phase 3 for the Da Nang Sentinel-2 assignment.

Manual requirements covered here: 1. Co-register the 2026 image onto the 2024 reference frame using feature detection, feature matching, RANSAC, and a homography. 2. Stitch the split AOI exports (`above` and `below`) back into a mosaic using the same feature-matching and homography-estimation pipeline.

The notebook exports all plots and summary files to `../output` with the prefix `phase3_`.

[15]:
```
import csv
import json
from pathlib import Path

import cv2
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image

# Paths are relative to this notebook when run from the works directory.
dataset_dir = Path("../dataset")
output_dir = Path("../output")
output_dir.mkdir(parents=True, exist_ok=True)

dates = {
    "2024": {"folder": dataset_dir / "2024_04_11", "suffix": "2024", "label":␣
 ↪"April 11, 2024"},
    "2026": {"folder": dataset_dir / "2026_04_11", "suffix": "2026", "label":␣
 ↪"April 11, 2026"},
}

crops = {
    "above": "Upper AOI crop",
    "below": "Lower AOI crop",
}

print(f"Dataset directory: {dataset_dir.resolve()}")
print(f"Output directory: {output_dir.resolve()}")
print(f"OpenCV version: {cv2.__version__}")
print(f"SIFT available: {hasattr(cv2, 'SIFT_create')}")
```

```
Dataset directory: /home/dacekey/CPV301_SUM26/main/L27_Assignment_*/dataset
Output directory: /home/dacekey/CPV301_SUM26/main/L27_Assignment_*/output
OpenCV version: 4.13.0
SIFT available: True
```

[16]:
```
def find_one(folder, pattern):
    matches = sorted(folder.glob(pattern))
    if not matches:
        raise FileNotFoundError(f"No file matching {pattern!r} in {folder}")
    if len(matches) > 1:
        print(f"Warning: multiple matches for {pattern!r} in {folder}; using␣
 ↪{matches[0].name}")
    return matches[0]


def crop_folder(date_key, crop_key):
```

```
    config = dates[date_key]
    return config["folder"] / f"{crop_key}_{config['suffix']}"


def normalize_to_uint8(image):
    return cv2.normalize(image, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)


def load_true_color(path):
    image = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if image is None:
        raise FileNotFoundError(f"Cannot load image: {path}")
    if image.ndim != 3 or image.shape[2] < 3:
        raise ValueError(f"Expected a 3-channel true-color image for {path},␣
 ↪got shape {image.shape}")
    rgb = cv2.cvtColor(image[:, :, :3], cv2.COLOR_BGR2RGB)
    return normalize_to_uint8(rgb)


def prepare_gray(rgb):
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
    return cv2.equalizeHist(gray)


# def save_figure(fig, filename):
    path = output_dir / filename
#     fig.savefig(path, dpi=200, bbox_inches="tight")
    print(f"Saved: {path}")
    return path


def save_rgb_image(image, filename):
    path = output_dir / filename
    Image.fromarray(image).save(path)
    print(f"Saved: {path}")
    return path


def create_sift():
    if not hasattr(cv2, "SIFT_create"):
        raise RuntimeError("This environment does not provide cv2.SIFT_create().
 ↪")
    return cv2.SIFT_create(nfeatures=3000, contrastThreshold=0.01,␣
 ↪edgeThreshold=10)


def reprojection_rmse(src_points, dst_points, homography, inlier_mask):
```

```
    if homography is None or inlier_mask is None:
        return float("nan")
    inliers = inlier_mask.ravel().astype(bool)
    if not np.any(inliers):
        return float("nan")
    projected = cv2.perspectiveTransform(src_points[inliers], homography)
    residual = projected.reshape(-1, 2) - dst_points[inliers].reshape(-1, 2)
    return float(np.sqrt(np.mean(np.sum(residual ** 2, axis=1))))


def estimate_homography(fixed_rgb, moving_rgb, ratio=0.75, ransac_threshold=5.
↪0):
    """Estimate a homography that maps moving_rgb coordinates into fixed_rgb␣
↪coordinates."""
    detector = create_sift()
    fixed_gray = prepare_gray(fixed_rgb)
    moving_gray = prepare_gray(moving_rgb)

    fixed_keypoints, fixed_desc = detector.detectAndCompute(fixed_gray, None)
    moving_keypoints, moving_desc = detector.detectAndCompute(moving_gray, None)
    if fixed_desc is None or moving_desc is None:
        raise RuntimeError("SIFT could not compute descriptors for one of the␣
↪images.")

    matcher = cv2.BFMatcher(cv2.NORM_L2)
    knn_matches = matcher.knnMatch(moving_desc, fixed_desc, k=2)

    good_matches = []
    for pair in knn_matches:
        if len(pair) != 2:
            continue
        m, n = pair
        if m.distance < ratio * n.distance:
            good_matches.append(m)

    if len(good_matches) < 4:
        raise RuntimeError(f"Need at least 4 good matches, got␣
↪{len(good_matches)}")

    src_points = np.float32([moving_keypoints[m.queryIdx].pt for m in␣
↪good_matches]).reshape(-1, 1, 2)
    dst_points = np.float32([fixed_keypoints[m.trainIdx].pt for m in␣
↪good_matches]).reshape(-1, 1, 2)
    homography, inlier_mask = cv2.findHomography(src_points, dst_points, cv2.
↪RANSAC, ransac_threshold)
    if homography is None:
```

```
        raise RuntimeError("RANSAC could not estimate a homography.")

    inliers = int(inlier_mask.sum())
    rmse = reprojection_rmse(src_points, dst_points, homography, inlier_mask)
    metrics = {
        "keypoints_fixed": len(fixed_keypoints),
        "keypoints_moving": len(moving_keypoints),
        "knn_matches": len(knn_matches),
        "good_matches": len(good_matches),
        "inliers": inliers,
        "inlier_ratio": float(inliers / len(good_matches)),
        "reprojection_rmse_px": rmse,
    }
    return {
        "homography": homography,
        "fixed_keypoints": fixed_keypoints,
        "moving_keypoints": moving_keypoints,
        "good_matches": good_matches,
        "inlier_mask": inlier_mask,
        "metrics": metrics,
        "src_points": src_points,
        "dst_points": dst_points,
    }


def inlier_matches(match_result):
    mask = match_result["inlier_mask"].ravel().astype(bool)
    return [m for m, keep in zip(match_result["good_matches"], mask) if keep]


def draw_inlier_matches(fixed_rgb, moving_rgb, match_result, max_matches=80):
    matches = inlier_matches(match_result)[:max_matches]
    return cv2.drawMatches(
        moving_rgb,
        match_result["moving_keypoints"],
        fixed_rgb,
        match_result["fixed_keypoints"],
        matches,
        None,
        matchColor=(0, 255, 0),
        singlePointColor=(255, 0, 0),
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS,
    )


def warp_to_fixed(moving_rgb, homography, fixed_shape):
    height, width = fixed_shape[:2]
```

```
    warped = cv2.warpPerspective(moving_rgb, homography, (width, height))
    mask = cv2.warpPerspective(np.ones(moving_rgb.shape[:2], dtype=np.uint8) *␣
↪255, homography, (width, height)) > 0
    return warped, mask


def registration_overlay(fixed_rgb, warped_rgb, valid_mask=None):
    fixed_gray = prepare_gray(fixed_rgb)
    warped_gray = prepare_gray(warped_rgb)
    overlay = np.zeros((*fixed_gray.shape, 3), dtype=np.uint8)
    overlay[:, :, 0] = fixed_gray
    overlay[:, :, 1] = warped_gray
    overlay[:, :, 2] = warped_gray
    if valid_mask is not None:
        overlay[~valid_mask] = 0
    return overlay


def difference_image(fixed_rgb, moving_rgb, mask=None):
    fixed_gray = prepare_gray(fixed_rgb)
    moving_gray = prepare_gray(moving_rgb)
    diff = cv2.absdiff(fixed_gray, moving_gray)
    if mask is not None:
        diff = diff.copy()
        diff[~mask] = 0
    return diff


def mean_abs_difference(fixed_rgb, moving_rgb, mask=None):
    diff = difference_image(fixed_rgb, moving_rgb, mask)
    if mask is not None and np.any(mask):
        return float(np.mean(diff[mask]))
    return float(np.mean(diff))


def stitch_with_homography(fixed_rgb, moving_rgb, homography):
    """Place fixed_rgb and moving_rgb on one canvas and feather-blend their␣
↪overlap."""
    h_fixed, w_fixed = fixed_rgb.shape[:2]
    h_moving, w_moving = moving_rgb.shape[:2]

    fixed_corners = np.float32([[0, 0], [w_fixed, 0], [w_fixed, h_fixed], [0,␣
↪h_fixed]]).reshape(-1, 1, 2)
    moving_corners = np.float32([[0, 0], [w_moving, 0], [w_moving, h_moving],␣
↪[0, h_moving]]).reshape(-1, 1, 2)
    moving_transformed = cv2.perspectiveTransform(moving_corners, homography)
    all_corners = np.vstack([fixed_corners, moving_transformed]).reshape(-1, 2)
```

```
    min_xy = np.floor(all_corners.min(axis=0)).astype(int)
    max_xy = np.ceil(all_corners.max(axis=0)).astype(int)
    translation = np.array([[1, 0, -min_xy[0]], [0, 1, -min_xy[1]], [0, 0, 1]],␣
↪dtype=np.float64)
    canvas_size = (int(max_xy[0] - min_xy[0]), int(max_xy[1] - min_xy[1]))

    fixed_warp = cv2.warpPerspective(fixed_rgb, translation, canvas_size)
    moving_warp = cv2.warpPerspective(moving_rgb, translation @ homography,␣
↪canvas_size)

    fixed_mask = cv2.warpPerspective(np.ones((h_fixed, w_fixed), dtype=np.
↪uint8) * 255, translation, canvas_size) > 0
    moving_mask = cv2.warpPerspective(np.ones((h_moving, w_moving), dtype=np.
↪uint8) * 255, translation @ homography, canvas_size) > 0

    fixed_distance = cv2.distanceTransform(fixed_mask.astype(np.uint8), cv2.
↪DIST_L2, 5)
    moving_distance = cv2.distanceTransform(moving_mask.astype(np.uint8), cv2.
↪DIST_L2, 5)
    fixed_distance[~fixed_mask] = 0
    moving_distance[~moving_mask] = 0
    denominator = fixed_distance + moving_distance

    mosaic = np.zeros_like(fixed_warp, dtype=np.float32)
    valid = denominator > 0
    mosaic[valid] = (
        fixed_warp[valid].astype(np.float32) * fixed_distance[valid, None]
        + moving_warp[valid].astype(np.float32) * moving_distance[valid, None]
    ) / denominator[valid, None]

    overlap_pixels = int(np.count_nonzero(fixed_mask & moving_mask))
    return {
        "mosaic": mosaic.astype(np.uint8),
        "fixed_warp": fixed_warp,
        "moving_warp": moving_warp,
        "fixed_mask": fixed_mask,
        "moving_mask": moving_mask,
        "translation": translation,
        "canvas_size": canvas_size,
        "overlap_pixels": overlap_pixels,
    }
```

[17]:
```
scene_data = {}

for date_key, date_info in dates.items():
    for crop_key, crop_label in crops.items():
```

```
        folder = crop_folder(date_key, crop_key)
        true_color_path = find_one(folder, "*True_color*.tiff")
        rgb = load_true_color(true_color_path)
        scene_data[(date_key, crop_key)] = {
            "rgb": rgb,
            "label": f"{crop_label} - {date_info['label']}",
            "path": true_color_path,
        }
        print(f"Loaded {date_key}/{crop_key}: shape={rgb.shape},␣
↪file={true_color_path.name}")
```

```
Loaded 2024/above: shape=(252, 774, 3),
file=2024-04-11-00_00_2024-04-11-23_59_Sentinel-2_L2A_True_color.tiff
Loaded 2024/below: shape=(313, 743, 3),
file=2024-04-11-00_00_2024-04-11-23_59_Sentinel-2_L2A_True_color.tiff
Loaded 2026/above: shape=(252, 774, 3),
file=2026-04-11-00_00_2026-04-11-23_59_Sentinel-2_L2A_True_color.tiff
Loaded 2026/below: shape=(313, 743, 3),
file=2026-04-11-00_00_2026-04-11-23_59_Sentinel-2_L2A_True_color.tiff
```

```
[ WARN:0@15.654] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@15.654] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@15.654] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@15.654] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@15.661] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@15.661] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@15.661] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@15.661] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@15.669] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@15.669] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@15.669] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@15.669] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@15.677] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@15.677] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
```

```
[ WARN:0@15.677] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@15.677] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
```

[18]:
```
coregistration_results = {}
coregistration_rows = []
homographies = {"coregistration": {}, "stitching": {}}

match_images = []
comparison_rows = []
difference_rows = []

for crop_key, crop_label in crops.items():
    fixed = scene_data[("2024", crop_key)]["rgb"]
    moving = scene_data[("2026", crop_key)]["rgb"]
    result = estimate_homography(fixed, moving)
    homography = result["homography"]
    warped, valid_mask = warp_to_fixed(moving, homography, fixed.shape)

    aligned_path = save_rgb_image(warped,␣
 ↪f"phase3_coregistered_2026_to_2024_{crop_key}.png")
    match_image = draw_inlier_matches(fixed, moving, result)
    overlay = registration_overlay(fixed, warped, valid_mask)
    before_diff = difference_image(fixed, moving)
    after_diff = difference_image(fixed, warped, valid_mask)
    before_mad = mean_abs_difference(fixed, moving)
    after_mad = mean_abs_difference(fixed, warped, valid_mask)

    metrics = result["metrics"]
    row = {
        "crop": crop_key,
        "fixed_date": "2024",
        "moving_date": "2026",
        "detector": "SIFT",
        "matcher": "BFMatcher_L2_ratio_0.75",
        "ransac_threshold_px": 5.0,
        "keypoints_fixed": metrics["keypoints_fixed"],
        "keypoints_moving": metrics["keypoints_moving"],
        "knn_matches": metrics["knn_matches"],
        "good_matches": metrics["good_matches"],
        "inliers": metrics["inliers"],
        "inlier_ratio": metrics["inlier_ratio"],
        "reprojection_rmse_px": metrics["reprojection_rmse_px"],
        "mean_abs_diff_before": before_mad,
        "mean_abs_diff_after": after_mad,
        "aligned_output": str(aligned_path),
```

```
    }
    coregistration_rows.append(row)
    coregistration_results[crop_key] = {
        "result": result,
        "warped": warped,
        "valid_mask": valid_mask,
        "overlay": overlay,
        "before_diff": before_diff,
        "after_diff": after_diff,
    }
    homographies["coregistration"][crop_key] = homography.tolist()

    match_images.append((crop_label, match_image, row))
    comparison_rows.append((crop_label, fixed, moving, warped, overlay))
    difference_rows.append((crop_label, before_diff, after_diff, row))

coreg_csv = output_dir / "phase3_coregistration_summary.csv"
with coreg_csv.open("w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=list(coregistration_rows[0].keys()))
    writer.writeheader()
    writer.writerows(coregistration_rows)
print(f"Saved: {coreg_csv}")

print("\nCo-registration summary")
for row in coregistration_rows:
    print(
        f"- {row['crop']}: good={row['good_matches']}, inliers={row['inliers']}␣
 ↪"
        f"({row['inlier_ratio']:.3f}), RMSE={row['reprojection_rmse_px']:.
 ↪3f}px, "
        f"MAD before/after={row['mean_abs_diff_before']:.2f}/
 ↪{row['mean_abs_diff_after']:.2f}"
    )

fig, axes = plt.subplots(len(match_images), 1, figsize=(16, 7),␣
 ↪constrained_layout=True)
if len(match_images) == 1:
    axes = [axes]
for ax, (crop_label, match_image, row) in zip(axes, match_images):
    ax.imshow(match_image)
    ax.set_title(
        f"{crop_label}: 2026 to 2024 SIFT inlier matches "
        f"({row['inliers']}/{row['good_matches']}, RMSE␣
 ↪{row['reprojection_rmse_px']:.2f}px)"
    )
    ax.axis("off")
fig.suptitle("Phase 3 Co-registration: RANSAC Inlier Matches", fontsize=14)
```

```
# save_figure(fig, "phase3_coregistration_matches_2026_to_2024.png")
plt.show()

fig, axes = plt.subplots(len(comparison_rows), 4, figsize=(18, 8),␣
 ↪constrained_layout=True)
for row_idx, (crop_label, fixed, moving, warped, overlay) in␣
 ↪enumerate(comparison_rows):
    panels = [
        (fixed, "2024 fixed reference"),
        (moving, "2026 original moving"),
        (warped, "2026 warped to 2024"),
        (overlay, "False-color overlay"),
    ]
    for col_idx, (image, title) in enumerate(panels):
        ax = axes[row_idx, col_idx]
        ax.imshow(image)
        ax.set_title(f"{crop_label}\n{title}")
        ax.axis("off")
fig.suptitle("Phase 3 Co-registration Result", fontsize=14)
# save_figure(fig, "phase3_coregistration_comparison_2026_to_2024.png")
plt.show()

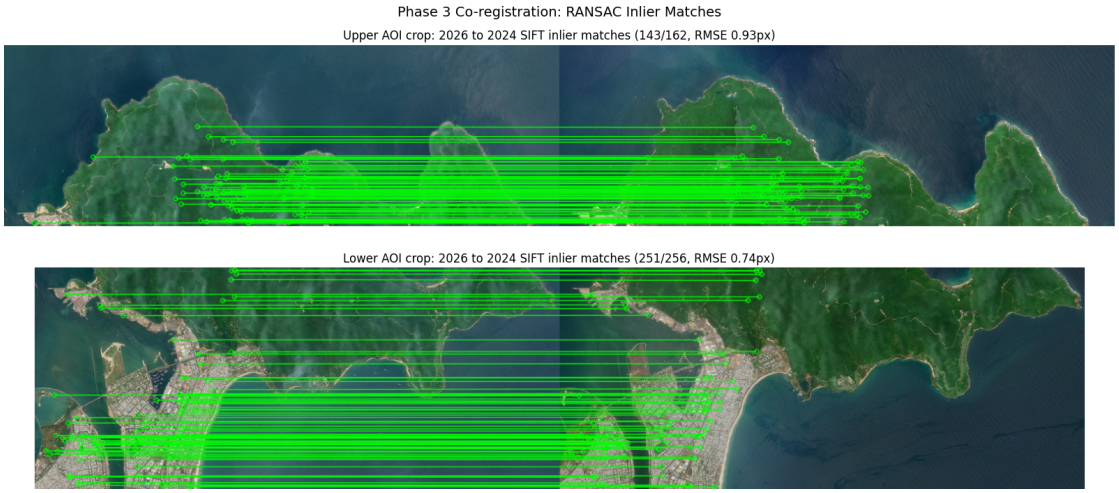
fig, axes = plt.subplots(len(difference_rows), 2, figsize=(12, 7),␣
 ↪constrained_layout=True)
for row_idx, (crop_label, before_diff, after_diff, row) in␣
 ↪enumerate(difference_rows):
    axes[row_idx, 0].imshow(before_diff, cmap="magma", vmin=0, vmax=120)
    axes[row_idx, 0].set_title(f"{crop_label}\nBefore alignment␣
 ↪MAD={row['mean_abs_diff_before']:.2f}")
    axes[row_idx, 1].imshow(after_diff, cmap="magma", vmin=0, vmax=120)
    axes[row_idx, 1].set_title(f"After alignment␣
 ↪MAD={row['mean_abs_diff_after']:.2f}")
    for col_idx in range(2):
        axes[row_idx, col_idx].axis("off")
fig.suptitle("Phase 3 Co-registration: Absolute Difference Before vs. After",␣
 ↪fontsize=14)
# save_figure(fig, "phase3_coregistration_difference_before_after.png")
plt.show()
```

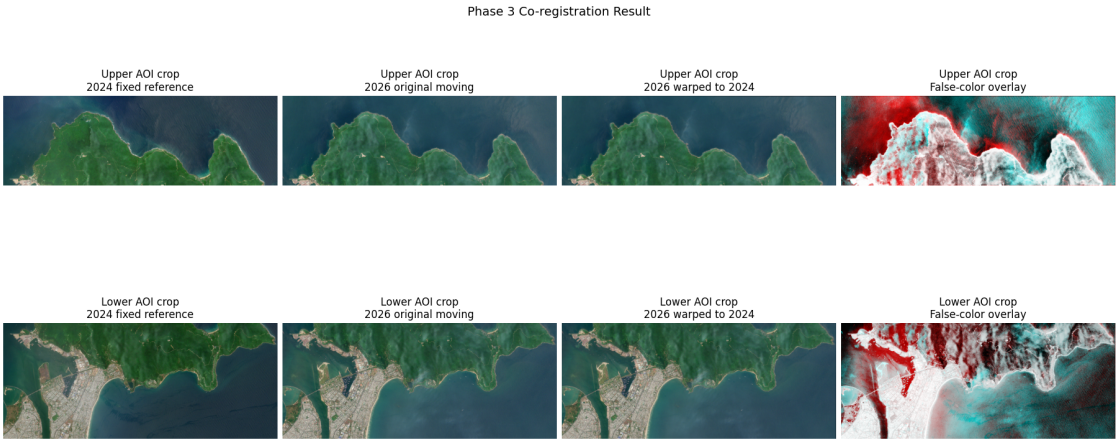
```
Saved: ../output/phase3_coregistered_2026_to_2024_above.png
```

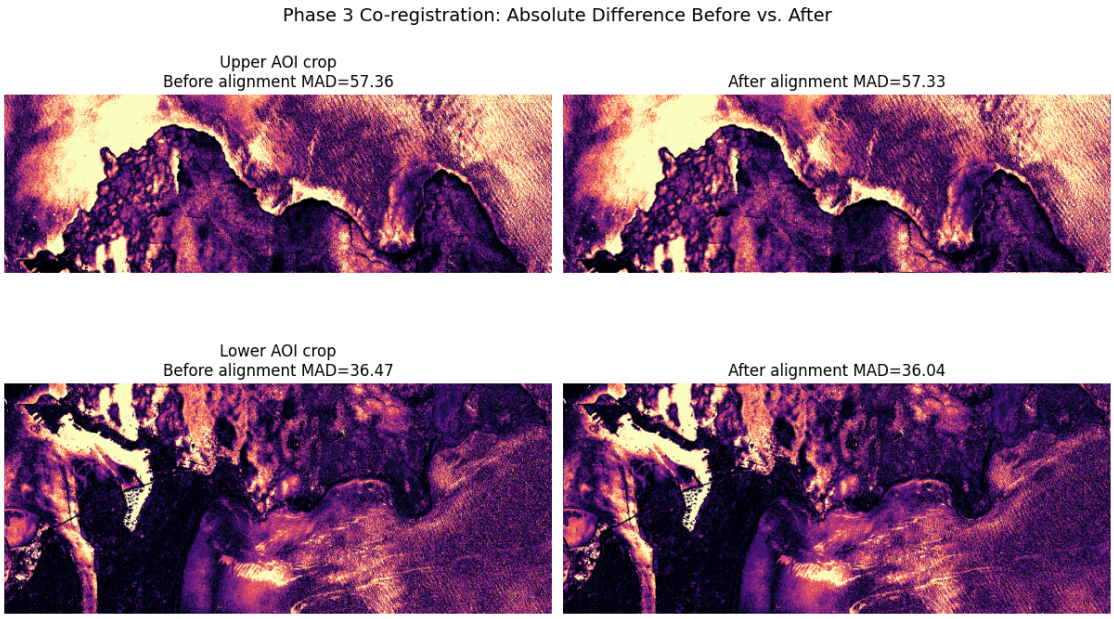
```
Saved: ../output/phase3_coregistered_2026_to_2024_below.png
Saved: ../output/phase3_coregistration_summary.csv

Co-registration summary
- above: good=162, inliers=143 (0.883), RMSE=0.934px, MAD
before/after=57.36/57.33
- below: good=256, inliers=251 (0.980), RMSE=0.741px, MAD
```

before/after=36.47/36.04







[19]:
```
stitching_results = {}
stitching_rows = []
stitch_match_images = []

for date_key, date_info in dates.items():
    fixed_above = scene_data[(date_key, "above")]["rgb"]
    moving_below = scene_data[(date_key, "below")]["rgb"]

    result = estimate_homography(fixed_above, moving_below)
    homography = result["homography"]
    stitch = stitch_with_homography(fixed_above, moving_below, homography)
    mosaic = stitch["mosaic"]

    mosaic_path = save_rgb_image(mosaic, f"phase3_stitched_mosaic_{date_key}.
 ↪png")
    match_image = draw_inlier_matches(fixed_above, moving_below, result)

    metrics = result["metrics"]
    row = {
        "date": date_key,
        "fixed_crop": "above",
        "moving_crop": "below",
        "detector": "SIFT",
        "matcher": "BFMatcher_L2_ratio_0.75",
        "ransac_threshold_px": 5.0,
        "keypoints_fixed": metrics["keypoints_fixed"],
```

```
        "keypoints_moving": metrics["keypoints_moving"],
        "knn_matches": metrics["knn_matches"],
        "good_matches": metrics["good_matches"],
        "inliers": metrics["inliers"],
        "inlier_ratio": metrics["inlier_ratio"],
        "reprojection_rmse_px": metrics["reprojection_rmse_px"],
        "canvas_width": stitch["canvas_size"][0],
        "canvas_height": stitch["canvas_size"][1],
        "overlap_pixels": stitch["overlap_pixels"],
        "mosaic_output": str(mosaic_path),
    }
    stitching_rows.append(row)
    stitching_results[date_key] = {
        "result": result,
        "stitch": stitch,
        "mosaic": mosaic,
    }
    homographies["stitching"][date_key] = homography.tolist()
    stitch_match_images.append((date_info["label"], match_image, row))

stitch_csv = output_dir / "phase3_stitching_summary.csv"
with stitch_csv.open("w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=list(stitching_rows[0].keys()))
    writer.writeheader()
    writer.writerows(stitching_rows)
print(f"Saved: {stitch_csv}")

homography_path = output_dir / "phase3_homographies.json"
with homography_path.open("w") as f:
    json.dump(homographies, f, indent=2)
print(f"Saved: {homography_path}")

print("\nStitching summary")
for row in stitching_rows:
    print(
        f"- {row['date']}: good={row['good_matches']}, inliers={row['inliers']}␣
 ↪"
        f"({row['inlier_ratio']:.3f}), RMSE={row['reprojection_rmse_px']:.
 ↪3f}px, "
        f"canvas={row['canvas_width']}x{row['canvas_height']},␣
 ↪overlap={row['overlap_pixels']} px"
    )

fig, axes = plt.subplots(len(stitch_match_images), 1, figsize=(16, 7),␣
 ↪constrained_layout=True)
if len(stitch_match_images) == 1:
    axes = [axes]
```

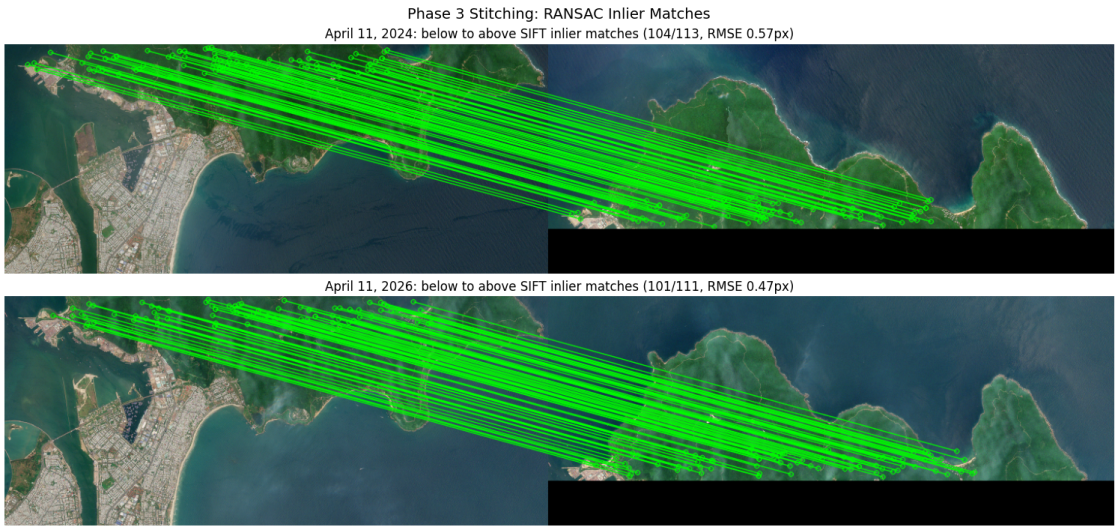
```
for ax, (date_label, match_image, row) in zip(axes, stitch_match_images):
    ax.imshow(match_image)
    ax.set_title(
        f"{date_label}: below to above SIFT inlier matches "
        f"({row['inliers']}/{row['good_matches']}, RMSE␣
 ↪{row['reprojection_rmse_px']:.2f}px)"
    )
    ax.axis("off")
fig.suptitle("Phase 3 Stitching: RANSAC Inlier Matches", fontsize=14)
# save_figure(fig, "phase3_stitching_matches_above_below.png")
plt.show()

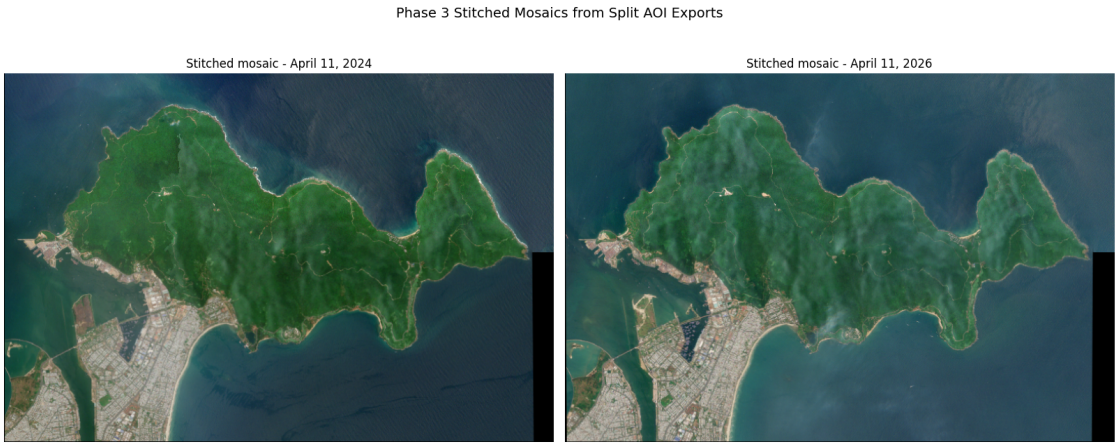
fig, axes = plt.subplots(1, 2, figsize=(16, 7), constrained_layout=True)
for ax, (date_key, date_info) in zip(axes, dates.items()):
    ax.imshow(stitching_results[date_key]["mosaic"])
    ax.set_title(f"Stitched mosaic - {date_info['label']}")
    ax.axis("off")
fig.suptitle("Phase 3 Stitched Mosaics from Split AOI Exports", fontsize=14)
# save_figure(fig, "phase3_stitched_mosaics_comparison.png")
plt.show()
```

```
Saved: ../output/phase3_stitched_mosaic_2024.png
```

```
Saved: ../output/phase3_stitched_mosaic_2026.png
Saved: ../output/phase3_stitching_summary.csv
Saved: ../output/phase3_homographies.json

Stitching summary
- 2024: good=113, inliers=104 (0.920), RMSE=0.573px, canvas=775x519,
overlap=35712 px
- 2026: good=111, inliers=101 (0.910), RMSE=0.473px, canvas=775x519,
overlap=35712 px
```





3.1 Phase 3 Notes

The co-registration step estimates one homography per crop to map the 2026 image into the 2024 coordinate frame. The homographies are close to identity because the exported Sentinel-2 crops already cover the same nominal geographic extents, but the feature/RANSAC step still verifies pixel-level consistency.

The stitching step estimates one homography per date to map the lower AOI crop onto the upper crop. The estimated vertical translation is around 204 pixels, which is consistent with the visible overlap between the two exported pieces. Feather blending is used in the overlap to reduce the seam caused by small radiometric differences between exports.

The generated summary CSV files and `phase3_homographies.json` can be reused by Phase 4 for change detection and tracking.

4 Phase 4: Detection and Change Tracking

This notebook implements Phase 4 for the Da Nang Sentinel-2 assignment.

Manual requirements covered here: 1. Detect and label a class of interest: built-up / urban-proxy regions. 2. Use Phase 3 co-registration homographies to warp 2026 onto the 2024 frame. 3. Compute difference maps and outline the strongest changed regions. 4. Discuss likely false-change sources and the controls used to reduce them.

All outputs are saved to `../output` with the prefix `phase4_`. This notebook does not modify Phase 1, Phase 2, Phase 3, or the report.

[20]:
```
import csv
import json
from pathlib import Path
```

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from matplotlib.patches import Rectangle

# Paths are relative to this notebook when run from the works directory.
dataset_dir = Path("../dataset")
output_dir = Path("../output")
output_dir.mkdir(parents=True, exist_ok=True)

homography_path = output_dir / "phase3_homographies.json"
if not homography_path.exists():
    raise FileNotFoundError(
        "Phase 4 needs Phase 3 homographies. Run works/cpvass_phase3.ipynb␣
 ↪first "
        "or provide ../output/phase3_homographies.json."
    )

with homography_path.open() as f:
    phase3_homographies = json.load(f)

dates = {
    "2024": {"folder": dataset_dir / "2024_04_11", "suffix": "2024", "label":␣
 ↪"April 11, 2024"},
    "2026": {"folder": dataset_dir / "2026_04_11", "suffix": "2026", "label":␣
 ↪"April 11, 2026"},
}

crops = {
    "above": "Upper AOI crop",
    "below": "Lower AOI crop",
}

print(f"Dataset directory: {dataset_dir.resolve()}")
print(f"Output directory: {output_dir.resolve()}")
print(f"Loaded Phase 3 homographies from: {homography_path}")
```

```
Dataset directory: /home/dacekey/CPV301_SUM26/main/L27_Assignment_*/dataset
Output directory: /home/dacekey/CPV301_SUM26/main/L27_Assignment_*/output
Loaded Phase 3 homographies from: ../output/phase3_homographies.json
```

[21]:
```
def find_one(folder, pattern):
    matches = sorted(folder.glob(pattern))
    if not matches:
        raise FileNotFoundError(f"No file matching {pattern!r} in {folder}")
    if len(matches) > 1:
```

```
        print(f"Warning: multiple matches for {pattern!r} in {folder}; using␣
 ↪{matches[0].name}")
    return matches[0]


def crop_folder(date_key, crop_key):
    config = dates[date_key]
    return config["folder"] / f"{crop_key}_{config['suffix']}"


def normalize_to_uint8(image):
    return cv2.normalize(image, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)


def load_true_color(path):
    image = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if image is None:
        raise FileNotFoundError(f"Cannot load image: {path}")
    if image.ndim != 3 or image.shape[2] < 3:
        raise ValueError(f"Expected a 3-channel true-color image for {path},␣
 ↪got shape {image.shape}")
    rgb = cv2.cvtColor(image[:, :, :3], cv2.COLOR_BGR2RGB)
    return normalize_to_uint8(rgb)


def load_band(path):
    band = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if band is None:
        raise FileNotFoundError(f"Cannot load image: {path}")
    if band.ndim != 2:
        raise ValueError(f"Expected single-band TIFF for {path}, got shape␣
 ↪{band.shape}")
    return band.astype(np.float32)


def load_scene(date_key, crop_key):
    folder = crop_folder(date_key, crop_key)
    rgb = load_true_color(find_one(folder, "*True_color*.tiff"))
    bands = {
        "B02": load_band(find_one(folder, "*B02*.tiff")),
        "B03": load_band(find_one(folder, "*B03*.tiff")),
        "B04": load_band(find_one(folder, "*B04*.tiff")),
        "B08": load_band(find_one(folder, "*B08*.tiff")),
    }
    for name, band in bands.items():
        if band.shape != rgb.shape[:2]:
```

```
            raise ValueError(f"Shape mismatch for {date_key}/{crop_key}/{name}:␣
 ↪{band.shape} vs {rgb.shape[:2]}")
    return {"rgb": rgb, "bands": bands}


def safe_index(numerator, denominator):
    return np.divide(
        numerator,
        denominator,
        out=np.zeros_like(numerator, dtype=np.float32),
        where=denominator != 0,
    )


def calculate_ndvi(red, nir):
    return safe_index(nir - red, nir + red)


def calculate_ndwi(green, nir):
    return safe_index(green - nir, green + nir)


def warp_scene_to_fixed(moving_scene, homography, fixed_shape):
    height, width = fixed_shape[:2]
    warped_rgb = cv2.warpPerspective(moving_scene["rgb"], homography, (width,␣
 ↪height))
    warped_bands = {
        name: cv2.warpPerspective(band, homography, (width, height))
        for name, band in moving_scene["bands"].items()
    }
    valid_mask = cv2.warpPerspective(
        np.ones(moving_scene["rgb"].shape[:2], dtype=np.uint8) * 255,
        homography,
        (width, height),
    ) > 0
    return {"rgb": warped_rgb, "bands": warped_bands, "valid_mask": valid_mask}


# def save_figure(fig, filename):
    path = output_dir / filename
#     fig.savefig(path, dpi=200, bbox_inches="tight")
    print(f"Saved: {path}")
    return path


def save_rgb_image(image, filename):
    path = output_dir / filename
```

```
    Image.fromarray(image).save(path)
    print(f"Saved: {path}")
    return path


def clean_binary_mask(mask, open_size=3, close_size=5):
    output = mask.astype(np.uint8)
    if open_size:
        output = cv2.morphologyEx(output, cv2.MORPH_OPEN, np.ones((open_size,␣
 ↪open_size), dtype=np.uint8))
    if close_size:
        output = cv2.morphologyEx(output, cv2.MORPH_CLOSE, np.ones((close_size,␣
 ↪close_size), dtype=np.uint8))
    return output.astype(bool)


def draw_mask_overlay(rgb, masks, alpha=0.45):
    overlay = rgb.copy().astype(np.float32)
    for mask, color in masks:
        if mask is None:
            continue
        overlay[mask] = (1 - alpha) * overlay[mask] + alpha * np.array(color,␣
 ↪dtype=np.float32)
    return np.clip(overlay, 0, 255).astype(np.uint8)


def draw_contours(rgb, mask, color=(255, 255, 0), thickness=2):
    output = rgb.copy()
    contours, _ = cv2.findContours(mask.astype(np.uint8), cv2.RETR_EXTERNAL,␣
 ↪cv2.CHAIN_APPROX_SIMPLE)
    cv2.drawContours(output, contours, -1, color, thickness)
    return output


def connected_regions(mask, min_area=60, max_regions=None):
    count, labels, stats, centroids = cv2.connectedComponentsWithStats(mask.
 ↪astype(np.uint8), connectivity=8)
    regions = []
    for label_id in range(1, count):
        area = int(stats[label_id, cv2.CC_STAT_AREA])
        if area < min_area:
            continue
        x = int(stats[label_id, cv2.CC_STAT_LEFT])
        y = int(stats[label_id, cv2.CC_STAT_TOP])
        w = int(stats[label_id, cv2.CC_STAT_WIDTH])
        h = int(stats[label_id, cv2.CC_STAT_HEIGHT])
        cx, cy = centroids[label_id]
```

```
        regions.append({
            "label_id": int(label_id),
            "area_px": area,
            "bbox_x": x,
            "bbox_y": y,
            "bbox_w": w,
            "bbox_h": h,
            "centroid_x": float(cx),
            "centroid_y": float(cy),
        })
    regions.sort(key=lambda r: r["area_px"], reverse=True)
    if max_regions is not None:
        regions = regions[:max_regions]
    return regions, labels
```

[22]:
```
label_names = {
    0: "Water",
    1: "Dense vegetation",
    2: "Built-up/urban proxy",
    3: "Sand/exposed soil",
    4: "Mixed/shadow/other",
}

label_colors = np.array([
    [37, 99, 235],     # water: blue
    [34, 139, 34],     # vegetation: green
    [220, 38, 38],     # built-up/urban proxy: red
    [245, 158, 11],    # sand/exposed soil: orange
    [107, 114, 128],   # mixed/shadow/other: gray
], dtype=np.uint8)


def classify_landcover(scene, valid_mask=None):
    rgb = scene["rgb"]
    bands = scene["bands"]
    red = bands["B04"]
    green = bands["B03"]
    nir = bands["B08"]

    ndvi = calculate_ndvi(red, nir)
    ndwi = calculate_ndwi(green, nir)
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
    hsv = cv2.cvtColor(rgb, cv2.COLOR_RGB2HSV)
    saturation = hsv[:, :, 1]

    if valid_mask is None:
        valid_mask = np.ones(gray.shape, dtype=bool)
```

```
    water = (((ndwi > 0.05) & (ndvi < 0.10)) | ((gray < 70) & (ndvi < 0.15))) &␣
↪valid_mask
    vegetation = (ndvi > 0.35) & valid_mask
    cloud = (gray > 195) & (saturation < 80) & (ndvi < 0.35) & valid_mask
    shadow = (gray < 38) & (ndvi < 0.30) & valid_mask

    built = (~water) & (~vegetation) & (ndvi < 0.25) & (gray > 110) &␣
↪(saturation < 130) & (~cloud) & (~shadow) & valid_mask
    sand = (~water) & (~vegetation) & (~built) & (ndvi > -0.25) & (ndvi < 0.20)␣
↪& (gray > 80) & (saturation < 100) & valid_mask

    labels = np.full(gray.shape, 4, dtype=np.uint8)
    labels[water] = 0
    labels[vegetation] = 1
    labels[built] = 2
    labels[sand] = 3
    labels[~valid_mask] = 4

    built_clean = clean_binary_mask(built, open_size=3, close_size=5)
    label_image = label_colors[labels]

    return {
        "ndvi": ndvi,
        "ndwi": ndwi,
        "gray": gray,
        "saturation": saturation,
        "valid_mask": valid_mask,
        "water": water,
        "vegetation": vegetation,
        "built": built_clean,
        "sand": sand,
        "cloud": cloud,
        "shadow": shadow,
        "labels": labels,
        "label_image": label_image,
    }

scene_data = {}
analysis_data = {}

for crop_key, crop_label in crops.items():
    fixed_2024 = load_scene("2024", crop_key)
    moving_2026 = load_scene("2026", crop_key)
    homography = np.array(phase3_homographies["coregistration"][crop_key],␣
↪dtype=np.float64)
```

```
    aligned_2026 = warp_scene_to_fixed(moving_2026, homography,␣
↪fixed_2024["rgb"].shape)

    scene_data[("2024", crop_key)] = fixed_2024
    scene_data[("2026_aligned", crop_key)] = aligned_2026

    analysis_data[("2024", crop_key)] = classify_landcover(fixed_2024)
    analysis_data[("2026_aligned", crop_key)] =␣
↪classify_landcover(aligned_2026, aligned_2026["valid_mask"])

    print(
        f"Loaded and co-registered {crop_label}: "
        f"2024 shape={fixed_2024['rgb'].shape}, aligned 2026␣
↪shape={aligned_2026['rgb'].shape}"
    )
```

```
Loaded and co-registered Upper AOI crop: 2024 shape=(252, 774, 3), aligned 2026
shape=(252, 774, 3)
Loaded and co-registered Lower AOI crop: 2024 shape=(313, 743, 3), aligned 2026
shape=(313, 743, 3)
```

```
[ WARN:0@19.535] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.535] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.535] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.535] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.544] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.544] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.544] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.544] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.548] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.548] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.548] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.548] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.552] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.552] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
```

```
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.552] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.552] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.555] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.555] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.555] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.555] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.559] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.559] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.559] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.559] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.566] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.566] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.566] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.566] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.569] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.569] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.569] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.569] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.572] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.572] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.572] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.572] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.575] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.575] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
```

```
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.575] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.575] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.609] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.609] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.609] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.609] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.620] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.620] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.620] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.620] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.624] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.624] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.624] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.624] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.629] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.629] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.629] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.629] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.633] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.633] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.633] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.633] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.637] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.637] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
```

```
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.637] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.637] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.646] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.646] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.646] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.646] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.650] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.650] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.650] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.650] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.654] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.654] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.654] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.654] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
[ WARN:0@19.658] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33550 (0x830e) encountered
[ WARN:0@19.658] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 33922 (0x8482) encountered
[ WARN:0@19.658] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34735 (0x87af) encountered
[ WARN:0@19.658] global grfmt_tiff.cpp:122 TIFF_Warning TIFFReadDirectory:
Unknown field with tag 34737 (0x87b1) encountered
```

[23]:
```
detection_rows = []
area_rows = []

fig, axes = plt.subplots(len(crops), 4, figsize=(18, 8),␣
 ↪constrained_layout=True)

for row_idx, (crop_key, crop_label) in enumerate(crops.items()):
    for date_col, date_key in enumerate(["2024", "2026_aligned"]):
        scene = scene_data[(date_key, crop_key)]
```

```
        analysis = analysis_data[(date_key, crop_key)]
        built = analysis["built"]
        valid = analysis["valid_mask"]
        total_valid = int(np.count_nonzero(valid))

        built_regions, built_labels = connected_regions(built, min_area=50,␣
↪max_regions=20)
        for idx, region in enumerate(built_regions, start=1):
            region_mask = built_labels == region["label_id"]
            detection_rows.append({
                "date": date_key,
                "crop": crop_key,
                "region_rank": idx,
                "area_px": region["area_px"],
                "bbox_x": region["bbox_x"],
                "bbox_y": region["bbox_y"],
                "bbox_w": region["bbox_w"],
                "bbox_h": region["bbox_h"],
                "centroid_x": region["centroid_x"],
                "centroid_y": region["centroid_y"],
                "mean_ndvi": float(np.mean(analysis["ndvi"][region_mask])),
                "mean_ndwi": float(np.mean(analysis["ndwi"][region_mask])),
            })

        for class_id, class_name in label_names.items():
            mask = (analysis["labels"] == class_id) & valid
            area_rows.append({
                "date": date_key,
                "crop": crop_key,
                "class_id": class_id,
                "class_name": class_name,
                "pixels": int(np.count_nonzero(mask)),
                "percent_valid": float(np.count_nonzero(mask) / total_valid *␣
↪100.0) if total_valid else 0.0,
            })

        overlay = draw_mask_overlay(scene["rgb"], [(built, (255, 0, 0))],␣
↪alpha=0.50)
        contour_overlay = draw_contours(overlay, built, color=(255, 255, 0),␣
↪thickness=2)
        label_image = analysis["label_image"]

        ax_overlay = axes[row_idx, date_col * 2]
        ax_labels = axes[row_idx, date_col * 2 + 1]
        title_date = "2024 reference" if date_key == "2024" else "2026 aligned␣
↪to 2024"
        ax_overlay.imshow(contour_overlay)
```

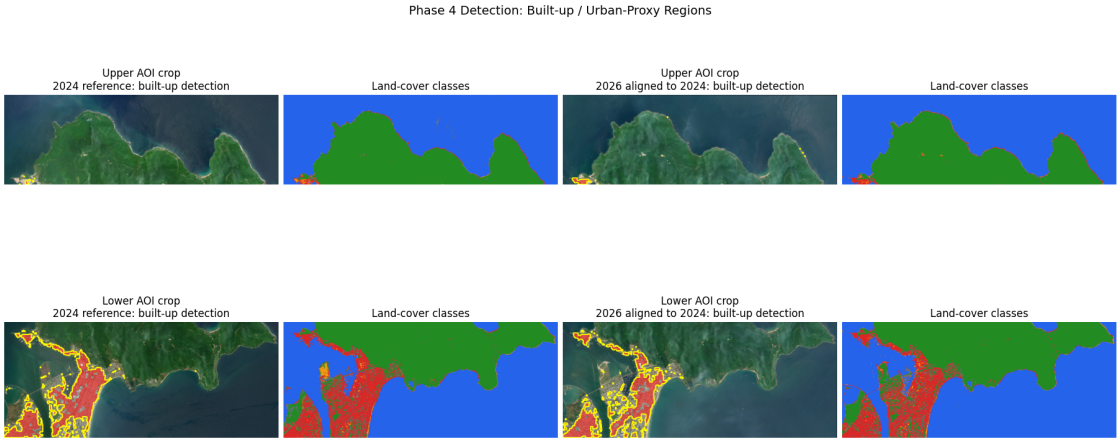
```
        ax_overlay.set_title(f"{crop_label}\n{title_date}: built-up detection")
        ax_labels.imshow(label_image)
        ax_labels.set_title("Land-cover classes")
        ax_overlay.axis("off")
        ax_labels.axis("off")

fig.suptitle("Phase 4 Detection: Built-up / Urban-Proxy Regions", fontsize=14)
# save_figure(fig, "phase4_builtup_detection_2024_2026.png")
plt.show()

regions_csv = output_dir / "phase4_builtup_detection_regions.csv"
with regions_csv.open("w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=list(detection_rows[0].keys()))
    writer.writeheader()
    writer.writerows(detection_rows)
print(f"Saved: {regions_csv}")

area_csv = output_dir / "phase4_landcover_area_summary.csv"
with area_csv.open("w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=list(area_rows[0].keys()))
    writer.writeheader()
    writer.writerows(area_rows)
print(f"Saved: {area_csv}")

print("\nBuilt-up detection summary")
for crop_key in crops:
    for date_key in ["2024", "2026_aligned"]:
        built_px = next(row["pixels"] for row in area_rows if row["crop"] ==␣
 ↪crop_key and row["date"] == date_key and row["class_id"] == 2)
        built_pct = next(row["percent_valid"] for row in area_rows if␣
 ↪row["crop"] == crop_key and row["date"] == date_key and row["class_id"] == 2)
        region_count = sum(1 for row in detection_rows if row["crop"] ==␣
 ↪crop_key and row["date"] == date_key)
        print(f"- {crop_key} / {date_key}: built-up={built_px} px ({built_pct:.
 ↪2f}%), labeled regions={region_count}")
```




```
Saved: ../output/phase4_builtup_detection_regions.csv
Saved: ../output/phase4_landcover_area_summary.csv

Built-up detection summary
- above / 2024: built-up=1471 px (0.75%), labeled regions=1
- above / 2026_aligned: built-up=1893 px (0.97%), labeled regions=1
- below / 2024: built-up=25971 px (11.17%), labeled regions=10
- below / 2026_aligned: built-up=24184 px (10.40%), labeled regions=18
```

[24]:
```
built_change_rows = []
fig, axes = plt.subplots(len(crops), 3, figsize=(16, 8),␣
 ↪constrained_layout=True)

for row_idx, (crop_key, crop_label) in enumerate(crops.items()):
    scene_2024 = scene_data[("2024", crop_key)]
    scene_2026 = scene_data[("2026_aligned", crop_key)]
    a2024 = analysis_data[("2024", crop_key)]
    a2026 = analysis_data[("2026_aligned", crop_key)]

    valid = a2026["valid_mask"] & ~(a2024["cloud"] | a2026["cloud"] |␣
 ↪a2024["shadow"] | a2026["shadow"])
    built_2024 = a2024["built"] & valid
    built_2026 = a2026["built"] & valid

    built_gain = clean_binary_mask(built_2026 & ~built_2024, open_size=3,␣
 ↪close_size=5)
    built_loss = clean_binary_mask(built_2024 & ~built_2026, open_size=3,␣
 ↪close_size=5)

    gain_regions, gain_labels = connected_regions(built_gain, min_area=50,␣
 ↪max_regions=20)
```

```
    loss_regions, loss_labels = connected_regions(built_loss, min_area=50,␣
↪max_regions=20)

    for change_type, regions, labels in [
        ("urban_proxy_gain", gain_regions, gain_labels),
        ("urban_proxy_loss_or_reclassification", loss_regions, loss_labels),
    ]:
        for idx, region in enumerate(regions, start=1):
            region_mask = labels == region["label_id"]
            built_change_rows.append({
                "crop": crop_key,
                "change_type": change_type,
                "region_rank": idx,
                "area_px": region["area_px"],
                "bbox_x": region["bbox_x"],
                "bbox_y": region["bbox_y"],
                "bbox_w": region["bbox_w"],
                "bbox_h": region["bbox_h"],
                "centroid_x": region["centroid_x"],
                "centroid_y": region["centroid_y"],
                "mean_ndvi_2024": float(np.mean(a2024["ndvi"][region_mask])),
                "mean_ndvi_2026_aligned": float(np.
↪mean(a2026["ndvi"][region_mask])),
                "mean_ndvi_delta": float(np.mean(a2026["ndvi"][region_mask] -␣
↪a2024["ndvi"][region_mask])),
            })

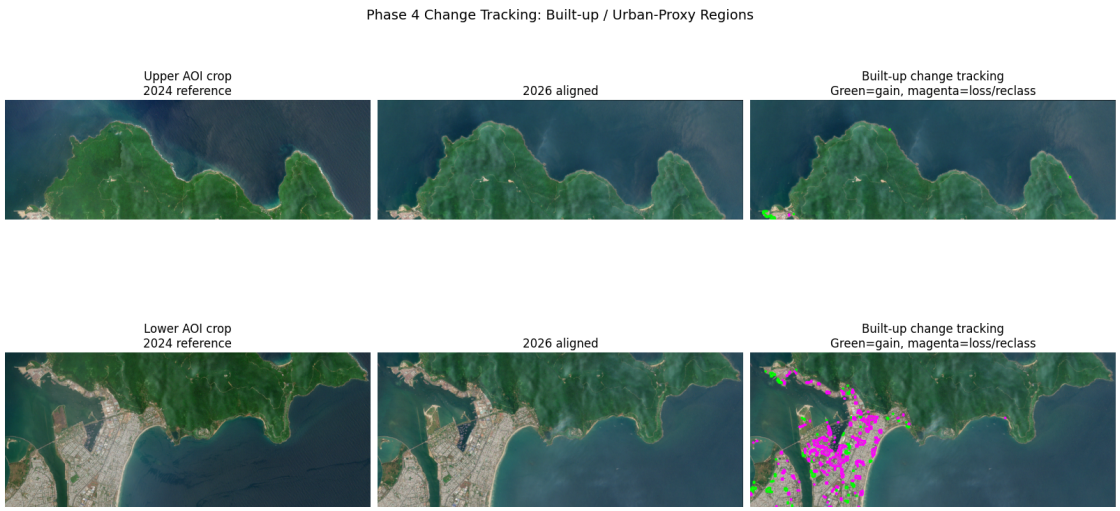
    overlay = draw_mask_overlay(
        scene_2026["rgb"],
        [(built_gain, (0, 255, 0)), (built_loss, (255, 0, 255))],
        alpha=0.60,
    )
    overlay = draw_contours(overlay, built_gain, color=(0, 255, 0), thickness=2)
    overlay = draw_contours(overlay, built_loss, color=(255, 0, 255),␣
↪thickness=2)

    axes[row_idx, 0].imshow(scene_2024["rgb"])
    axes[row_idx, 0].set_title(f"{crop_label}\n2024 reference")
    axes[row_idx, 1].imshow(scene_2026["rgb"])
    axes[row_idx, 1].set_title("2026 aligned")
    axes[row_idx, 2].imshow(overlay)
    axes[row_idx, 2].set_title("Built-up change tracking\nGreen=gain,␣
↪magenta=loss/reclass")
    for col_idx in range(3):
        axes[row_idx, col_idx].axis("off")
```

```
fig.suptitle("Phase 4 Change Tracking: Built-up / Urban-Proxy Regions",␣
 ↪fontsize=14)
# save_figure(fig, "phase4_builtup_change_tracking.png")
plt.show()

built_change_csv = output_dir / "phase4_builtup_change_regions.csv"
if built_change_rows:
    with built_change_csv.open("w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=list(built_change_rows[0].keys()))
        writer.writeheader()
        writer.writerows(built_change_rows)
else:
    with built_change_csv.open("w", newline="") as f:
        f.
 ↪write("crop,change_type,region_rank,area_px,bbox_x,bbox_y,bbox_w,bbox_h,centroid_x,centroid
print(f"Saved: {built_change_csv}")

print("\nBuilt-up change tracking summary")
for crop_key in crops:
    gain_count = sum(1 for row in built_change_rows if row["crop"] == crop_key␣
 ↪and row["change_type"] == "urban_proxy_gain")
    loss_count = sum(1 for row in built_change_rows if row["crop"] == crop_key␣
 ↪and row["change_type"] == "urban_proxy_loss_or_reclassification")
    gain_area = sum(row["area_px"] for row in built_change_rows if row["crop"]␣
 ↪== crop_key and row["change_type"] == "urban_proxy_gain")
    loss_area = sum(row["area_px"] for row in built_change_rows if row["crop"]␣
 ↪== crop_key and row["change_type"] == "urban_proxy_loss_or_reclassification")
    print(f"- {crop_key}: gain regions={gain_count} ({gain_area} px), loss/
 ↪reclass regions={loss_count} ({loss_area} px)")
```



```
Saved: ../output/phase4_builtup_change_regions.csv

Built-up change tracking summary
- above: gain regions=1 (180 px), loss/reclass regions=0 (0 px)
- below: gain regions=8 (588 px), loss/reclass regions=14 (3029 px)
```

[25]:
```
change_rows = []
change_maps = {}
fig_maps, axes_maps = plt.subplots(len(crops), 4, figsize=(18, 8),␣
 ↪constrained_layout=True)
fig_regions, axes_regions = plt.subplots(len(crops), 3, figsize=(16, 8),␣
 ↪constrained_layout=True)

for row_idx, (crop_key, crop_label) in enumerate(crops.items()):
    scene_2024 = scene_data[("2024", crop_key)]
    scene_2026 = scene_data[("2026_aligned", crop_key)]
    a2024 = analysis_data[("2024", crop_key)]
    a2026 = analysis_data[("2026_aligned", crop_key)]

    valid = a2026["valid_mask"]
    land = valid & ~(a2024["water"] & a2026["water"])
    clean_valid = land & ~(a2024["cloud"] | a2026["cloud"] | a2024["shadow"] |␣
 ↪a2026["shadow"])

    ndvi_delta = a2026["ndvi"] - a2024["ndvi"]
    ndwi_delta = a2026["ndwi"] - a2024["ndwi"]
    gray_delta = a2026["gray"].astype(np.float32) - a2024["gray"].astype(np.
 ↪float32)

    change_score = (
        0.45 * np.abs(ndvi_delta)
        + 0.35 * np.abs(gray_delta) / 255.0
        + 0.20 * np.abs(ndwi_delta)
    )
    threshold = float(np.percentile(change_score[clean_valid], 95)) if np.
 ↪any(clean_valid) else float("nan")
    strong_change = (change_score >= threshold) & clean_valid
    strong_change = clean_binary_mask(strong_change, open_size=3, close_size=5)

    regions, labels = connected_regions(strong_change, min_area=50,␣
 ↪max_regions=8)
    change_maps[crop_key] = {
        "change_score": change_score,
        "threshold": threshold,
        "strong_change": strong_change,
        "labels": labels,
        "regions": regions,
```

```
        "clean_valid": clean_valid,
    }

    for idx, region in enumerate(regions, start=1):
        region_mask = labels == region["label_id"]
        built_2024_pct = float(np.count_nonzero(a2024["built"] & region_mask) /␣
↪region["area_px"] * 100.0)
        built_2026_pct = float(np.count_nonzero(a2026["built"] & region_mask) /␣
↪region["area_px"] * 100.0)
        water_2024_pct = float(np.count_nonzero(a2024["water"] & region_mask) /␣
↪region["area_px"] * 100.0)
        water_2026_pct = float(np.count_nonzero(a2026["water"] & region_mask) /␣
↪region["area_px"] * 100.0)
        mean_ndvi_delta = float(np.mean(ndvi_delta[region_mask]))
        mean_score = float(np.mean(change_score[region_mask]))

        if built_2026_pct - built_2024_pct > 20:
            interpretation = "urban-proxy gain candidate"
        elif built_2024_pct - built_2026_pct > 20:
            interpretation = "urban-proxy loss/reclassification candidate"
        elif abs(water_2026_pct - water_2024_pct) > 20:
            interpretation = "water/shoreline or turbidity change candidate"
        elif abs(mean_ndvi_delta) > 0.15:
            interpretation = "vegetation spectral change candidate"
        else:
            interpretation = "mixed spectral change candidate"

        change_rows.append({
            "crop": crop_key,
            "region_rank": idx,
            "area_px": region["area_px"],
            "bbox_x": region["bbox_x"],
            "bbox_y": region["bbox_y"],
            "bbox_w": region["bbox_w"],
            "bbox_h": region["bbox_h"],
            "centroid_x": region["centroid_x"],
            "centroid_y": region["centroid_y"],
            "mean_change_score": mean_score,
            "mean_ndvi_2024": float(np.mean(a2024["ndvi"][region_mask])),
            "mean_ndvi_2026_aligned": float(np.
↪mean(a2026["ndvi"][region_mask])),
            "mean_ndvi_delta": mean_ndvi_delta,
            "built_pct_2024": built_2024_pct,
            "built_pct_2026_aligned": built_2026_pct,
            "water_pct_2024": water_2024_pct,
            "water_pct_2026_aligned": water_2026_pct,
            "interpretation": interpretation,
```

```
        })

    # Difference-map panel.
    axes_maps[row_idx, 0].imshow(ndvi_delta, cmap="RdYlGn", vmin=-0.5, vmax=0.5)
    axes_maps[row_idx, 0].set_title(f"{crop_label}\nNDVI delta")
    axes_maps[row_idx, 1].imshow(np.abs(gray_delta), cmap="magma", vmin=0,␣
 ↪vmax=120)
    axes_maps[row_idx, 1].set_title("Brightness absolute delta")
    axes_maps[row_idx, 2].imshow(change_score, cmap="inferno", vmin=0,␣
 ↪vmax=max(0.30, threshold))
    axes_maps[row_idx, 2].set_title(f"Change score\n95th pct={threshold:.3f}")
    axes_maps[row_idx, 3].imshow(strong_change, cmap="gray")
    axes_maps[row_idx, 3].set_title("Strong-change mask")
    for col_idx in range(4):
        axes_maps[row_idx, col_idx].axis("off")

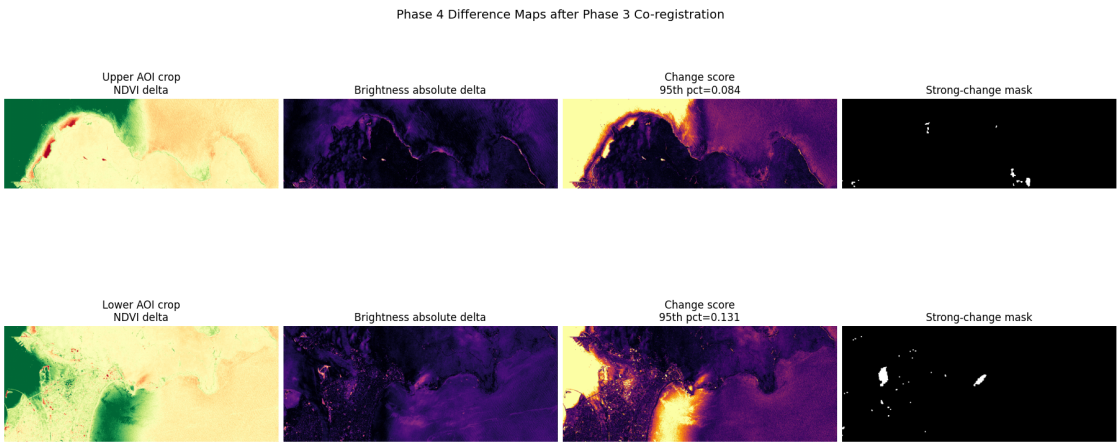
    # Region tracking panel.
    outlined_2024 = scene_2024["rgb"].copy()
    outlined_2026 = scene_2026["rgb"].copy()
    change_overlay = draw_mask_overlay(scene_2026["rgb"], [(strong_change,␣
 ↪(255, 255, 0))], alpha=0.60)
    for region in regions[:5]:
        x, y, w, h = region["bbox_x"], region["bbox_y"], region["bbox_w"],␣
 ↪region["bbox_h"]
        cv2.rectangle(outlined_2024, (x, y), (x + w, y + h), (255, 255, 0), 2)
        cv2.rectangle(outlined_2026, (x, y), (x + w, y + h), (255, 255, 0), 2)
        cv2.rectangle(change_overlay, (x, y), (x + w, y + h), (255, 255, 0), 2)
    axes_regions[row_idx, 0].imshow(outlined_2024)
    axes_regions[row_idx, 0].set_title(f"{crop_label}\nTracked ROI in 2024")
    axes_regions[row_idx, 1].imshow(outlined_2026)
    axes_regions[row_idx, 1].set_title("Same ROI in aligned 2026")
    axes_regions[row_idx, 2].imshow(change_overlay)
    axes_regions[row_idx, 2].set_title("Outlined strongest changes")
    for col_idx in range(3):
        axes_regions[row_idx, col_idx].axis("off")

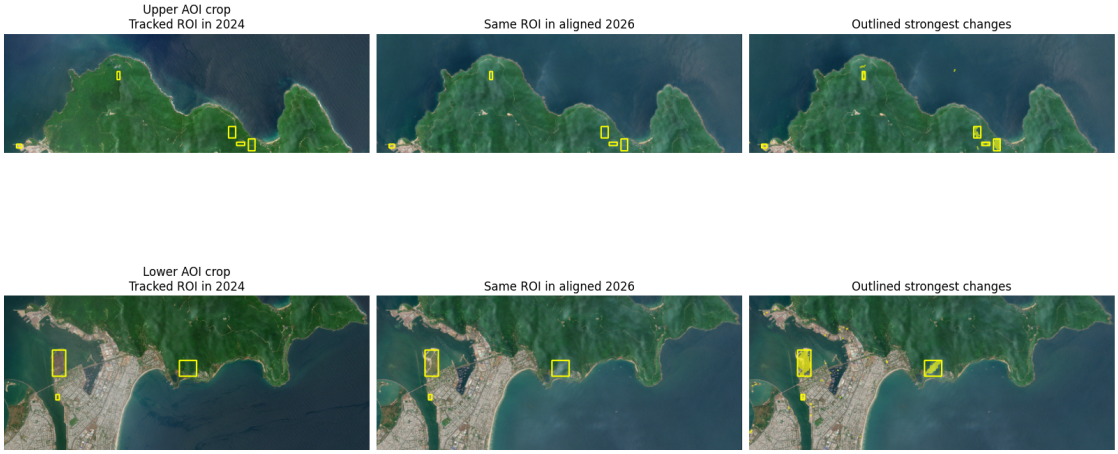
fig_maps.suptitle("Phase 4 Difference Maps after Phase 3 Co-registration",␣
 ↪fontsize=14)
# save_figure(fig_maps, "phase4_difference_maps.png")
plt.show()

fig_regions.suptitle("Phase 4 Change Tracking: Same ROIs across Ordered␣
 ↪Satellite Frames", fontsize=14)
# save_figure(fig_regions, "phase4_top_change_regions_tracking.png")
plt.show()
```

```
change_csv = output_dir / "phase4_top_change_regions.csv"
if change_rows:
    with change_csv.open("w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=list(change_rows[0].keys()))
        writer.writeheader()
        writer.writerows(change_rows)
else:
    with change_csv.open("w", newline="") as f:
        f.
 ↪write("crop,region_rank,area_px,bbox_x,bbox_y,bbox_w,bbox_h,centroid_x,centroid_y,mean_chan
print(f"Saved: {change_csv}")

print("\nTop change-region summary")
for row in change_rows[:12]:
    print(
        f"- {row['crop']} R{row['region_rank']}: area={row['area_px']} px, "
        f"score={row['mean_change_score']:.3f}, NDVI␣
 ↪delta={row['mean_ndvi_delta']:+.3f}, "
        f"built {row['built_pct_2024']:.1f}%->{row['built_pct_2026_aligned']:.
 ↪1f}%, "
        f"{row['interpretation']}"
    )
```






```
Saved: ../output/phase4_top_change_regions.csv

Top change-region summary
- above R1: area=247 px, score=0.097, NDVI delta=-0.091, built 0.0%->0.0%, mixed
spectral change candidate
- above R2: area=166 px, score=0.096, NDVI delta=-0.090, built 0.0%->0.0%, mixed
spectral change candidate
- above R3: area=67 px, score=0.100, NDVI delta=-0.056, built 0.0%->0.0%, mixed
spectral change candidate
- above R4: area=67 px, score=0.097, NDVI delta=-0.089, built 0.0%->0.0%, mixed
spectral change candidate
- above R5: area=53 px, score=0.181, NDVI delta=+0.217, built 0.0%->100.0%,
urban-proxy gain candidate
- above R6: area=51 px, score=0.169, NDVI delta=+0.176, built 0.0%->0.0%,
vegetation spectral change candidate
- below R1: area=903 px, score=0.237, NDVI delta=+0.317, built 3.8%->2.9%,
vegetation spectral change candidate
- below R2: area=413 px, score=0.181, NDVI delta=-0.167, built 0.0%->0.0%,
vegetation spectral change candidate
- below R3: area=54 px, score=0.243, NDVI delta=-0.277, built 0.0%->75.9%,
urban-proxy gain candidate
```

[26]:
```
control_rows = []
fig, axes = plt.subplots(len(crops), 4, figsize=(18, 8),␣
 ↪constrained_layout=True)

for row_idx, (crop_key, crop_label) in enumerate(crops.items()):
    a2024 = analysis_data[("2024", crop_key)]
    a2026 = analysis_data[("2026_aligned", crop_key)]
    scene_2026 = scene_data[("2026_aligned", crop_key)]
```

```
    valid = a2026["valid_mask"]
    cloud_shadow = (a2024["cloud"] | a2026["cloud"] | a2024["shadow"] |␣
 ↪a2026["shadow"]) & valid
    both_water = (a2024["water"] & a2026["water"]) & valid
    clean_valid = change_maps[crop_key]["clean_valid"]
    strong_change = change_maps[crop_key]["strong_change"]

    control_rows.append({
        "crop": crop_key,
        "valid_pixels": int(np.count_nonzero(valid)),
        "cloud_shadow_excluded_pixels": int(np.count_nonzero(cloud_shadow)),
        "both_water_excluded_pixels": int(np.count_nonzero(both_water)),
        "clean_land_pixels": int(np.count_nonzero(clean_valid)),
        "strong_change_pixels": int(np.count_nonzero(strong_change)),
        "strong_change_percent_of_clean_land": float(np.
 ↪count_nonzero(strong_change) / np.count_nonzero(clean_valid) * 100.0) if np.
 ↪any(clean_valid) else 0.0,
    })

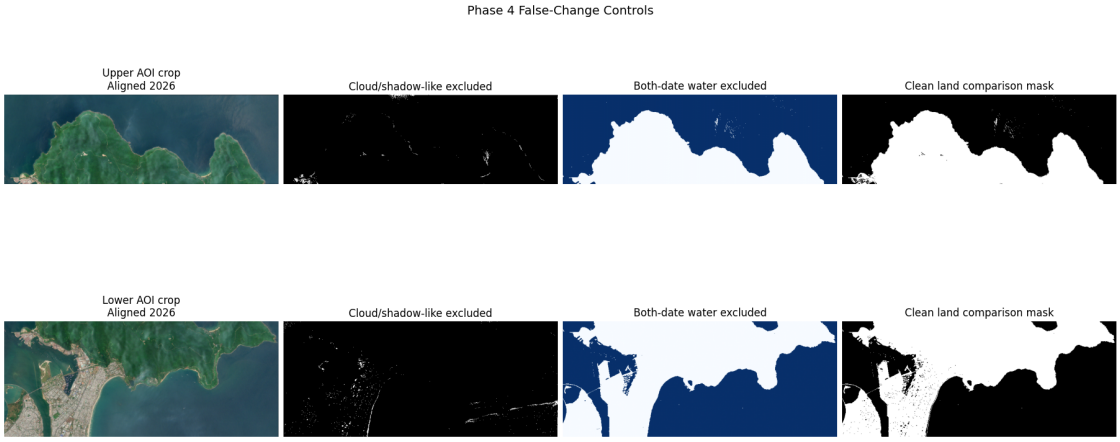
    axes[row_idx, 0].imshow(scene_2026["rgb"])
    axes[row_idx, 0].set_title(f"{crop_label}\nAligned 2026")
    axes[row_idx, 1].imshow(cloud_shadow, cmap="gray")
    axes[row_idx, 1].set_title("Cloud/shadow-like excluded")
    axes[row_idx, 2].imshow(both_water, cmap="Blues")
    axes[row_idx, 2].set_title("Both-date water excluded")
    axes[row_idx, 3].imshow(clean_valid, cmap="gray")
    axes[row_idx, 3].set_title("Clean land comparison mask")
    for col_idx in range(4):
        axes[row_idx, col_idx].axis("off")

fig.suptitle("Phase 4 False-Change Controls", fontsize=14)
# save_figure(fig, "phase4_false_change_controls.png")
plt.show()

control_csv = output_dir / "phase4_false_change_controls.csv"
with control_csv.open("w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=list(control_rows[0].keys()))
    writer.writeheader()
    writer.writerows(control_rows)
print(f"Saved: {control_csv}")

print("\nFalse-change control summary")
for row in control_rows:
    print(
        f"- {row['crop']}: clean land={row['clean_land_pixels']} px, "
        f"strong change={row['strong_change_pixels']} px "
        f"({row['strong_change_percent_of_clean_land']:.2f}% of clean land), "
```

```
        f"cloud/shadow excluded={row['cloud_shadow_excluded_pixels']} px"
    )
```




```
Saved: ../output/phase4_false_change_controls.csv

False-change control summary
- above: clean land=74511 px, strong change=722 px (0.97% of clean land),
cloud/shadow excluded=1420 px
- below: clean land=110836 px, strong change=1665 px (1.50% of clean land),
cloud/shadow excluded=1665 px
```

4.1 Phase 4 Notes

The detection target is built-up / urban-proxy land cover. Because only visible, Red, Green, Blue, and NIR bands are available, this is not a full urban classifier using SWIR/NDBI. The detector reuses the Phase 2 segmentation logic: water is suppressed with NDWI and dark-water rules, vegetation is suppressed with NDVI, and bright low-to-moderate saturation non-water/non-vegetation pixels are labeled as built-up/urban proxy.

Change tracking is treated as the satellite analogue of object tracking: each candidate changed region is represented as a fixed ROI in the 2024 coordinate frame, then the same ROI is inspected in the aligned 2026 frame. The notebook reports class percentages, NDVI delta, and a change interpretation for each tracked ROI.

Likely false-change sources are cloud/haze, shadows, water turbidity, sun-angle/radiometric differences, and small residual registration errors. Controls used here include same-month image dates, Phase 3 SIFT/RANSAC co-registration, exclusion of both-date water, exclusion of simple cloud/shadow-like pixels, connected-component filtering, and reporting candidate regions rather than treating every pixel difference as a real land-cover change.